

# Changelog

All notable changes to the Claude multi-account toolkit.

## v1.17.0 — 2026-07-18 — Cross-alias session & background-agent continuity

### Fixed

- **A session left under one alias was invisible from every other alias.** Opening a project from xiaomi, leaving the session with work in progress, and then opening deepseek (or any other alias) showed nothing — two independent root causes, both fixed and live-proven:
  1. **Session resolution applied only to the native transport's bare launches.** Every router alias (kimi-\*, poe, openrouter, chutes...) always opened a FRESH session, and alias -p "." on any transport skipped resolution entirely ("explicit args win verbatim"). **Fix:** `_cma_session_flags` now runs before the transport split — both transports resume — and conversation args get `--resume <sid>` injected unless the user already chose a session (`--resume/--session-id/ --continue/-c/--fork-session`) or invoked a non-conversation subcommand (agents, mcp, export, ...). Injection uses the new `claude-session existing-id`, which returns a session only when one actually EXISTS — the older `latest-id` falls back to a deterministic UUID for never-used projects, and resuming that fails hard with "No conversation found with session ID". **Live proof:** xiaomi (native) created a session in a test project; kimi-k3 (router) and deepseek (native) both resumed the identical session id and answered from its memory.
  2. **Background agents were registered per config dir.** Claude Code's background-agent registry (`daemon/roster.json + dispatch/`, plus the `jobs/` store) was local to each alias dir, so an agent started under xiaomi was invisible to deepseek. **Fix:** `daemon` and `jobs` are now shared items (`CMA_SHARED_ITEMS + unify's SHARED_ITEMS`, drift guard enforced), `daemon/roster.json` is **union-merged** by `cma_union_rosters` (newer `updatedAt` wins per worker — a per-file last-wins would drop other aliases' workers), and `cma_migrate_daemon_dirs_once` merges every existing provider `daemon` dir into the shared store (roster content stashed

before the backup move — the first cut of the migration unioned paths that had already moved), replaces it with the shared symlink, and is idempotent via a marker file. Live-verified: rosters from four provider dirs unioned into one shared registry; all provider daemon dirs are now symlinks.

## Added

- **Testing:** scripts/tests/test\_session\_flags.sh (12 assertions — both transports resume on bare launch, -p injection, no double-injection, subcommand passthrough, empty-session case, migration trigger); test\_unify.sh daemon section (roster union semantics, dir linking, jobs, provider-dir migration with backup + idempotency — 107 assertions in the file). Challenges/HelixQA: check 7 in provider\_aliases\_challenge.sh (shared items, union merge, flags placement, existing-id, live daemon symlinks — 23/23 PASS live) and bank case cma-pav-cross-alias-session-continuity (bank now 12 cases).

## v1.16.0 — 2026-07-18 — Output-token cap for router providers

### Fixed

- **“Claude’s response exceeded the 128000 output token maximum” on router providers.**  
CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS was exported **only on the native transport path** — every router provider (kimi-k3, poe, openrouter, ...) launched with Claude Code’s generic default output cap (128000 for models it does not know), so long reasoning responses died with that API error. **Fix:** the cap is now exported for **both** transports, before the transport split (`_cma_out_guard`), valued from the provider model’s real `limit.output` (`CMA_PROVIDER_MAX_OUTPUT`, e.g. 131072 for k3). Live-verified on the real wrapper: a kimi-k3 router launch now carries `CLAUDE_CODE_MAX_OUTPUT_TOKENS=131072`.
- **Catalog-conflation guard:** models.dev sometimes reports `limit.output == limit.context` (the “output” number is really the context size) or an inflated `limit.context` (nvidia5’s llama-3.2-11b-vision: catalog claims 1M context; the provider’s own error says 131072). Exporting such a cap makes Claude Code request more completion tokens than the shared window allows → 400 maximum context length ... you requested N. The guard exports **only a genuinely separate output budget** (`output < context`). nvidia5 was removed (its catalog metadata is wrong on both axes; backed up — self-heals via `sync --multi`).
- **Wrapper migration now covers the output-cap fix** — `_cma_out_guard` was added to the `cma_run_provider` self-heal

marker chain, so wrappers written before v1.16.0 regenerate automatically on the next `install.sh/sync/shell` start (otherwise hosts would keep the native-only export indefinitely).

- **e2e tool-call flake:** `alias_e2e_test.py`'s tool probe gets the same one-retry policy as the layer-1 verifier (weak free models occasionally skip an instructed tool call — `opcode2` false-FAILED a proof leg).
- **Router launch prompted interactively on some shells.** The wrapper's `ccr-config mv` ran as a bare command; on hosts whose interactive shell aliases `mv='mv -i'` (Fedora/RHEL defaults) the alias launch stopped at `mv: overwrite '.../config.json'? and hung or died on redirected stdin. All 16 mv sites in lib.sh/emitted wrappers are now command mv -f (bypasses aliases and shell functions, forces the overwrite).`
- **Cryptic failure when another tool named `ccr` shadows the router.** On a host where `ccr` resolved to a different program (a CCS-style profile manager), `ccr` code meant "launch profile 'code'" and the alias died with `Profile "code" was not found or is disabled`. The router path now verifies `ccr` version names the real `@musistudio/claude-code-router` and refuses early with an actionable message (fix `PATH` / remove the shadowing `ccr` / install command).

## Added

- **Testing:** new hermetic suite `scripts/tests/test_output_tokens.sh` (8 assertions — native + router export, empty-limit case, guard-before-split structure, migration trigger). Challenges/HelixQA: new bank case `cma-pav-output-cap-both-transport` and Check 6 in `provider_aliases_challenge.sh` (16/16 PASS live). Full suite: 24/24 files ALL GREEN; 6-leg proof ALL GREEN.

## v1.15.0 — 2026-07-18 — Full Kimi variant support (OAuth subscription models)

### Added

- **Every Kimi model the OAuth subscription serves is now a launchable alias.** `detect_kimicode_record` discovers the served models live (`GET {base}/models` with the OAuth token, unioned with the `models.dev` catalog since the listing under-reports) and emits one provider record per model — the old code exposed a single hardcoded alias. On this host: `kimi-for-coding` (account default, "K2.7 Coding"), `kimi-for-coding-highspeed`, `kimi-k2p7` (Kimi 2.7), and **`kimi-k3` (Kimi 3 — 1M context, reasoning)**. Every record goes through the same strict

sentinel + tool-calling + semantic verification as every other provider.

- **kimi\_proxy.py** — moonshot-flavored schema normalizer routed under every kimi-\* alias (new <family>\_proxy.py discovery rule in the launch wrapper). Model k3 rejects any tool whose parameters carries a \$ref not starting with #/\$defs/ (400 ... not a valid moonshot flavored json schema, reproduced live); Claude Code's tool schemas trip exactly that. The proxy hoists \$defs+definitions, rewrites foreign \$refs by last segment, and guarantees parameters.type/properties. Live proof: direct request → 400, same request through the proxy → 200.
- **Launch-time OAuth token freshness** (lib.sh emitted wrapper). The OAuth token lives ~15 minutes, so the old sync-time snapshot 401'd by the next launch — the root of “kimi-for-coding works once, then dies”. Freshness order at every launch: unexpired live credentials file → CLI-triggered refresh (kimi -p hi) → token-file snapshot (last resort).
- **API-key paths for kimi.com coding** — KIMI\_API\_KEY (catalog env) and ApiKey\_Kimi (new key-aliases.json entry) both resolve to the kimi-for-coding provider as a fallback for hosts without an OAuth session. OAuth subscription records take **precedence** over API-key records (unique\_by merge, detector first) — the subscription is the priority path.
- **sarvam\_proxy.py** — Sarvam compatibility proxy (same family-discovery mechanism). Three distinct runtime incompatibilities were root-caused and fixed, each reproduced live first: system/user message content arrays (400 ... Input should be a valid string — flattened to joined strings) and Claude Code's 64000-token output default exceeding the starter tier's 4096 cap (max\_tokens now clamped, overridable via SARVAM\_MAX\_OUTPUT\_TOKENS). Result: the sarvam alias went from guaranteed-400 at launch to a real Claude Code PASS.
- **Challenges/HelixQA**: three new bank cases (cma-pav-kimi-oauth-token-freshness, cma-pav-kimi-multi-model-oauth-records, cma-pav-kimi-k3-moonshot-schema-proxy) and Check 5 in provider\_aliases\_challenge.sh (detector discovery, precedence, freshness, schema proxy, live kimi-alias freshness) — 15/15 PASS live.

## Fixed

- **claude-providers verify <id> was unusable for OAuth providers** — it never injected the OAuth token, so verify-by-id always degraded to a false unverified. It now applies the same live-cred-file-first freshness order.
- **Layer-1 probes false-FAIL reasoning models** — 128-token budget was consumed entirely by chain-of-thought (k3, deepseek-v4-pro), yielding “empty content / sentinel missing” failures on working models. Probe budget is now 512 tokens.

- **Detector jq ARG\_MAX overflow** — the full models.dev catalog was passed as a `--argjson` argument; only the kimi-for-coding models subtree is passed now (the bug silently yielded zero OAuth records and let an API-key record shadow the subscription).
- **Wrapper self-heal migration did not cover the new wrapper features** — the `cma_run_provider` migration markers stopped at v1.14.0, so every host upgrading kept a stale wrapper that (a) never started `kimi_proxy` for `kimi-*` (k3 400'd on every tool call — live-confirmed) and (b) never refreshed the OAuth token (401 after ~15 min). The `_family_id` and `kimi-code/credentials/kimi-code.json` markers now trigger regeneration.
- **Live legs blind to OAuth + overcounted account states** — `verify_aliases_live.sh` silently skipped OAuth aliases ("no key"), had no CLI-refresh fallback (stale snapshots → 400/401), lacked the family proxy discovery (`kimi $ref` tests 400'd), used 32/64-token budgets that false-FAIL reasoning models (poe's claude-sonnet-4.6 needs 512 to reach the tool call — proven live), and FAILED on account limits (weekly cap, fair-use 1313) instead of classifying them. `alias_e2e_test.py` had the same OAuth-key and staleness gaps. Both legs now: resolve the OAuth token through the full live-cred → CLI-refresh → snapshot chain, discover family proxies, use 256/512 budgets, and classify **SKIP-QUOTA** / **SKIP-TRANSIENT** / **SKIP-GATED** (aliases the gate already filtered) honestly instead of as passes or failures.

## Testing

- New hermetic suite `scripts/tests/test_kimi.sh` (**33 assertions**): detector multi-record emission (live discovery + catalog union + offline fallback), expired-token CLI refresh, OAuth-first precedence + no duplicates, API-key fallback mapping, launch-time token freshness (all three sources), `cmd_verify` OAuth injection, `kimi_proxy` schema normalization (5 cases), family proxy discovery markers. Full suite: 22/22 files ALL GREEN.
- Live (Kimi aliases FIRST, per release gate): all 4 aliases verified by layer-1 probes, layer-3 semantic (sentinel + independent judge), and real Claude Code launches through the aliases.

# v1.14.0 — 2026-07-17 — Anti-bluff provider verification (strict filtering)

## Fixed

- **Provider aliases passed verification while being broken at runtime.** Aliases such as huggingface were marked verified yet any real prompt (“Do you see my codebase?”) returned API failures (402 depleted credits, suspended accounts, unsupported models). A live sweep showed **12 of 19 aliases failing while status.json called them verified. Root causes (all fixed):**
  1. *Layer-1 existence check was a bare GET /v1/models.* HTTP 200 proved only that the key was accepted — never inference, the selected model, or tool calling. **Fix:** `scripts/providers-verify.sh` now runs two live probes against the provider’s chat endpoint with the exact alias model: a sentinel probe (response MUST contain `VERIFY_OK`; 200-without-sentinel or error-in-200 is a bluff ⇒ failed) and a tool-calling probe (the model MUST emit a real tool call — Claude Code is tool-driven). Definitive rejections (401/402/403/404) ⇒ failed; only transient conditions (429/5xx/timeout/no-network) ⇒ unverified. Anthropic-native endpoints are probed in their native `/v1/messages` shape.
  2. *Multi path (`sync --multi`) verified chat-only models.* `model_verify.py` never asserted the `VERIFY_OK` sentinel it requested, set `verified=True` unconditionally, and counted tool calling as zero required points (`MIN_SCORE=25 ==` existence weight alone). **Fix:** sentinel is asserted (missing ⇒ anti-bluff failure), verified now **requires** a passed tool-calling probe, and the verification cache carries `_cache_version` so results from the old logic are never replayed.
  3. *Billing/auth failures were classed as transient.* The semantic code-visibility layer exited 3 (“infra — honest SKIP, never downgrade”) on HTTP 402/401, so a credits-dead provider kept its stale verified. **Fix (LLMsVerifier submodule):** semantic-code-visibility now maps 401/402/403/404 on model-under-test calls to exit 1 (genuine negative — demotes), keeps 429/5xx/timeout as exit 3, and keeps judge-call failures always at exit 3 (a broken judge never demotes the model under test).
  4. *The live alias verifier ignored tool calling.* `verify_aliases_live.sh` recorded “no tool call” but never failed on it — aliases passed with tool-less models. **Fix:** test 6 now uses an instructed tool call and is verdict-relevant.
  5. *The runtime-shaped e2e test was orphaned.* `alias_e2e_test.py` (tools, \$ref, cache\_control, streaming through the real endpoint) was never invoked by anything.

- Fix:** wired into `run-proof.sh` as `leg 44` with an honest `SKIP` (exit 3) when no providers/network are present.
6. *Probe URLs were mis-normalized for real provider bases.* Anthropic-native bases had their `/anthropic` prefix stripped (DeepSeek/Xiaomi 404'd on `.../v1/messages` when the served path is `.../anthropic/v1/messages`), and already-versioned bases (`.../paas/v4` on Z.AI/BigModel coding plans) got a bogus `/v1` inserted. **Fix:** the prefix is kept for native probes, versioned bases get only `/chat/completions` appended — in both `providers-verify.sh` and `LLMsVerifier`'s semantic-code-visibility (`chatCompletionsURL` rule, 11 new Go tests).
  7. *Single-attempt probes flapped on transient conditions.* Load-balanced gateways return occasional 400/404/412/000, and weaker models non-deterministically miss the sentinel or skip a tool call — working providers (kilo, siliconflow, sarvam) flipped to failed between syncs. **Fix:** exactly one retry for flappy codes and for flaky sentinel/tool-call outcomes; auth/billing codes (401/402/403) and consistent bluffs are never retried.
  8. *Layer-4 superpowers-TUI marker was model-phrasing-dependent.* Genuinely engaged sessions whose model summarized the framework in its own words (instead of the exact `superpowers:<name> announce` string) were failed. **Fix:** the marker also accepts vocabulary that can only exist when the skill content loaded (systematic-debugging, brainstorming, the `invoke-before-response` rule) — echo/refusal bluffs still cannot pass — and any transcript with `"is_error":true` is a hard FAIL (real API errors can't masquerade as engagement).
  9. *Poe aliases failed real Claude Code launches through the proxy.* `poe_proxy.py` injected parameters only when missing/null, but Poe actually requires the `properties` key — Claude Code's zero-argument tools (`{"type":"object"}` with no `properties`) were rejected with the misleading 400 `Invalid 'tools': Field required` (root cause reproduced and verified live against `api.poe.com`). **Fix:** `fix_tools` guarantees `parameters.properties` (and `type:object`) on every tool.
  10. *The live legs produced false FAILs on account/provider states and native transports.* `verify_aliases_live.sh` sent OpenAI-shaped Bearer requests to `/anthropic` bases (deepseek/xiaomi hung or 400'd) and had no retries, so single 000/429 wobbles failed working aliases; `alias_e2e_test.py` gave reasoning models (deepseek-v4-pro) a 128-token budget that reliably came back empty, and double-appended `/chat/completions` on versioned bases. **Fix:** both legs speak native Anthropic shape under the kept `/anthropic` prefix, versioned bases get exactly one `/chat/completions`, reasoning models get a 512-token budget with one empty-answer retry, and both legs now classify honestly: **SKIP-QUOTA** (account funds: 402/`insufficient_quota/...`) and **SKIP-TRANSIENT** (provider capacity/timeouts/5xx) are reported separately and never

counted as passes or toolkit failures — the same FUNDS distinction `verify_claude_live.sh` already made. Genuine FAIL (auth, schema, bluff, no tool call after retry) stays FAIL.

11. *A live API key leaked into committed proof evidence.*

`opcode debug config` embeds MCP server env (`"TAVILY_API_KEY": "tvly-dev-..."`), and `cma_redact_secrets` only matched keys literally named `apiKey|api_key|password|secret|token` — env-style names and the `tvly-/nvapi-` prefixes slipped through into `proof/10-debug-config.json`.

**Fix:** the redactor now matches any JSON key NAME containing `key/token/secret/password/api-key` and covers the extra prefixes (`${VAR}` placeholders still preserved); extended hermetic tests in `test_coverage.sh`; full re-scan of `proof/` is clean.

## Added

- **Tier C constitution/conformance verifier** `scripts/tests/verify_constitution.sh` (design spec §7.4, implemented at `scripts/tests/` alongside the other live verifiers): CONST-051 submodule decoupling, §11.4.157 four-file doc lockstep, §11.4.113 no force-push, §11.4.156 CI/CD disabled, §11.4.151 release-tag prefix, toolkit-owned fixture/rubric independence. Wired into `run-proof.sh` as leg 45 — the proof suite is now six legs: sandbox tier A, OpenCode live, providers live, aliases live, alias e2e, constitution tier C.
- Hermetic test coverage: `test_verify_scripts.sh` grew from ~30 to 69 assertions (sentinel/tools/402/429/bluff/Anthropic-shape/cache-version); new `test_constitution.sh`; `test_providers.sh` loopback servers now answer the real chat+tools probe shapes.
- **Challenges submodule:** new live anti-bluff challenge `challenges/scripts/provider_aliases_challenge.sh` (static pipeline-honesty checks + `status.json` freshness/consistency, SKIP-OK when the toolkit is absent) and HelixQA-compatible `bank_banks/examples/provider-alias-verification.json` (6 test cases), both registered in the `docs/test-coverage.md` ledger (describe-runner 25/25).

## Changed

- **Live filtering results (2026-07-17, all layers + real Claude Code launches):** every installed alias was re-verified with the strict pipeline and the ones that could not pass were removed (config dirs backed up as `~/.claude-prov-  
<id>.preunify.*`; re-adding is just `claude-providers sync` once the account is fixed). Removed: `huggingface (+2,3)`, `openrouter5`, `github-models`, `fireworks-ai`, `inference`, `novita-ai`, `sarvam`, `tencent-tokenhub`, `upstage`, `orphans zai/zhipuai`, stale `chutes4`,



xiaomi2/xiaomi3, rate-limited/unusable zai-coding-plan and zhipuai-coding-plan (429 fair-use limited / all-models-404 right now — honest removal, self-heals on the next sync), plus runtime-broken kimi-for-coding2 (k3 rejects Claude Code's \$ref tool schemas) and poe3 (gemma-4-31b aborts streams), and openrouter4 (nvidia/nemotron-nano-12b-v2-vl returns empty/malformed responses). **Final state: 28 aliases installed, every one verified through the full strict pipeline** (statuses, env files, and alias lines exactly consistent). overrides.json pins were corrected to models verified live today (openrouter, nvidia added; xiaomi fast model fixed to a served one).

- **LLMsVerifier submodule — strict scoring, no more bluff headroom:** removed the hard VerificationScore = max(score, 0.7) floor (every strict gate was vacuous); CodeVisibility confidence threshold 0.3 → 0.5 with rebalanced weights (a bare “Yes, I can see it” now scores 0.4 < 0.5); keyword matching uses \b word boundaries (“no” no longer matches “know”/“not”); round-1 sentinel check gained a prompt-echo guard (≥60-char verbatim fixture slice in the reply = genuine fail); the dead hardcoded HuggingFace endpoint api-inference.huggingface.co was replaced with https://router.huggingface.co/v1 in all production paths. Full Go suite: 59 packages green.
- docs/Provider\_Aliases\_User\_Guide.md updated to the new verification semantics (§3 command table, §5 multi-alias verification, §7 rewritten).

## v1.13.3 — 2026-07-17 — Session-sharing pipefail fix

### Fixed

- **All aliases created separate, unshared sessions for the same project.** Switching between claude1, claude2, deepseek, xiaomi, etc. in the same project directory started a fresh conversation instead of resuming the shared one — no memory, context, or history continuity across aliases. **Root cause:** claude-session.sh has set -o pipefail. When cma\_latest\_session\_id() scanned for existing sessions with `ls -t ... | grep | head -1`, head closed stdin after one line, sending SIGPIPE to grep. With pipefail, the pipeline exited 141, and set -e aborted the script BEFORE it could return the session UUID. Every launch was treated as a first run, creating a new random-UUID session. **Fix:** Added `|| true` guard on the head -1 pipeline in cma\_latest\_session\_id() (scripts/claude-session.sh line 111). The guard catches SIGPIPE without affecting the captured output — head already printed the first line before exiting, so latest gets the correct UUID. All aliases

now resolve to the same `--resume <uuid>`. **Verified with 200-file stress test** that exercises the exact pipefail condition.

## Added

- 200-file stress test in `test_session.sh` that proves `claude-session flags` returns `--resume` (not `--session-id`) even with many session files, and that `latest-id` returns the most recent session.
- Investigation document at `docs/investigations/2026-07-17-session-sharing-root-cause.md` with full root cause analysis, evidence, and architecture notes.

## v1.13.0 — 2026-07-10 — Project-scoped cwd-hook (session-resumption fix)

### Fixed

- **Sessions were resumed for the wrong project when switching Claude Code aliases.** When the global `~/.local/bin/claude-cwd-hook` symlink pointed to one project's multitrack resolver (e.g. `atmosphere's`), every `claudeN` alias launch was redirected into that project's worktree BEFORE `claude-session` resolved the session — so the session was keyed to the `atmosphere` track, not the project the user was actually working in (e.g. `helix_ota`). Switching from `claude4` to `claude1` resumed an `atmosphere Track-4` session instead of the `helix_ota` session. **Root cause:** `CMA_CWD_HOOK` was a single global singleton with no per-project awareness; the hook fired unconditionally and the toolkit had no mechanism for a repo to supply its own resolver. **Fix:** `cma_run` now resolves the `cwd-hook` in a three-tier order:
  1. `CMA_CWD_HOOK` env var (explicit override, unchanged),
  2. `<git-toplevel>/.claude-cwd-hook` (per-project hook — each repo gets its own multitrack resolver; prints nothing → stay in `$PWD`),
  3. `~/.local/bin/claude-cwd-hook` (global fallback, backward-compatible). A repo that needs its own track layout drops a `.claude-cwd-hook` at its git root; a repo that doesn't simply omits it and keeps the global hook. The migration self-heals outdated `cma_run` wrappers via a new `_cma_hook_root` marker (detected on next `install.sh` / shell start).

### Added

- Regression tests in `test_lib.sh` and `test_wrapper_exec.sh` prove the emitted `cma_run` body carries the project-scoped hook resolution code (`_cma_hook_root` marker, `git rev-parse --show-`

toplevel, .claude-cwd-hook check) and respects the CMA\_CWD\_HOOK override + global fallback.

## v1.12.3 — 2026-07-05 — Session-name sanitization (kebab-case)

### Changed

- **Auto-derived session names are now sanitized to kebab-case.** `claude-session.sh` derives the session name from the project directory's basename. It now: lowercases the name; trims leading/trailing whitespace; collapses internal whitespace and underscores to -; strips any remaining characters that are not `[a-z0-9-]`; and collapses consecutive - before trimming leading/trailing -. This makes session names safe for the CLI and filesystem while remaining human-readable.

### Added

- New regression tests in `test_session.sh` cover leading/trailing whitespace, multiple consecutive spaces, and stripping of special invalid characters.

## v1.12.2 — 2026-07-05 — Native alias auto-registration + account-detection hardening

### Fixed

- **Native `claude<N>` aliases were never created for pre-existing account dirs.** Running `install.sh` or `claude-unify.sh` only merged shared state; if account directories already existed (e.g. `~/.claude-milos85vasic`), no shell aliases were registered, so `claude1/claude2/etc.` were undefined. `claude-unify.sh` now auto-registers a `claude<N>` alias for every detected account that lacks one.
- **Smart alias numbering:** an account dir literally named `~/.claude-claude4` keeps the `claude4` alias; remaining dirs fill the lowest free `claude<N>` slot instead of skipping over reserved numbers.
- **Bogus account detection:** `~/.claude-code-router` and `~/.claude-*.lock` directories are no longer treated as Claude accounts, preventing them from stealing alias slots or being merged into shared state.

## Added

- Regression tests in `test_unify.sh` and `test_install.sh` prove that `install/unify` register native aliases for existing account dirs and preserve `claude<N>` basenames while filling gaps.

## v1.12.1 — 2026-07-05 — Judge independence + resolve/robustness hardening

Addresses the v1.12.0 final whole-branch review's deferred items and the deep-research findings on LLM-as-judge bias.

## Changed

- **Round-2 judge default is now a DIFFERENT model family.** `providers/judge.env.template` defaults to Groq / Llama-3.1 (`llama-3.1-8b-instant`) instead of DeepSeek. 2024-2026 research (arXiv:2508.06709 and others) shows a judge systematically favors its own model family — including validating that family's *wrong* answers, the exact failure layer-3 exists to catch — so defaulting the judge to a common subject (DeepSeek) was the worst case. Verified working live as an independent judge.
- **The semantic command distinguishes transport/infra failures from genuine verdicts.** The `LLMsVerifier semantic-code-visibility` command now exits **3** when a round-1/round-2 API call cannot complete (non-2xx, timeout, empty, connection error), vs exit **1** for a real negative verdict. `providers-semantic.sh` maps exit 3 → skip, so a transient judge/model hiccup is an honest SKIP and never downgrades the model-under-test (final-review I-1).

## Added

- **Independence warning:** `providers-semantic.sh` warns (never fails) when the judge endpoint equals the model-under-test endpoint (same provider = same family = not independent).
- **xAI is now resolvable:** `providers/overrides.json` gains `xai` → `https://api.x.ai/v1` (the catalog lists xAI with no API base, so it previously resolved unmapped). Endpoint confirmed live.
- CONST-069 capability-boundary mandate in the `LLMsVerifier` submodule constitution (records the CONST-051 project-agnostic boundary under a non-colliding id).

## Fixed

- A directory passed as the keys file (CMA\_KEYS\_FILE / --keys-file) now dies with a clear message instead of silently yielding “0 key vars”.

## v1.12.0 — 2026-07-05 — Semantic code-visibility (layer 3) + live TUI verification (layer 4)

Adds two new provider-verification layers on top of the Phase-1 existence/tool-call checks, driven by the LLMsVerifier submodule’s standalone, stdlib-only semantic-code-visibility Go command. The whole pipeline was proven end-to-end against real providers (redacted, real network): a genuinely code-seeing model (mistral-medium-2604) verifies (round-1 sentinel + round-2 judge 2/3); models that bluff or fail round-2 (chutes GLM-5.2-TEE — empty/timeout) are unverified; billing blocks (deepseek 402) are unverified — never a faked pass.

## Added

- **Layer 3 — semantic code-visibility.** scripts/providers-semantic.sh renders the toolkit-owned rubric (providers/rubric/code-visibility-rubric.json) into a judge prompt and drives the Go command through a build-and-cache driver (scripts/claude-semantic-visibility.sh). Two rounds: a unique sentinel embedded in a code fixture (does the model actually see the code?), then an independent LLM-as-judge score ( $\geq$  threshold). One-word contract verified|unverified|skip (exit 0/1/2). Wired into cmd\_sync; keys move via env only (never argv). CONST-051 boundary held — the submodule stays project-agnostic, receiving fixture/prompt/ judge-prompt/sentinel only as CLI args.
- **Layer 4 — live superpowers-TUI verification.** scripts/verify\_superpowers\_tui.sh launches real Claude Code through a provider alias and confirms the superpowers plugin engages with no trust/overwrite prompt — the only thing that flips a provider to fully verified. Honest SKIP when preconditions (real claude/key/network) are absent; the engagement classifier is hardened against false-PASS on prompt-echo, and a live negative-case test (neutral prompt → marker must NOT match) proved it does not false-verify.
- **claude-providers verify <id> [--deep]** — single-provider deep re-verify.
- **Tier-B live verifier** (scripts/tests/verify\_providers\_live.sh) runs layers 3-4 per installed provider, writes secret-redacted

evidence + an aggregate proof/providers-summary.json; already wired into run-proof.sh.

## Fixed

- **--refresh-aliases was not byte-idempotent** —  
cma\_provider\_write\_alias re-ran the full cma\_ensure\_alias\_file self-heal migrations on *every* alias line, which could non-deterministically reposition the cma\_run\_provider function and produce a different file on a second refresh (the session hook runs this on every shell). Now only bootstraps the alias file when absent. (Root-caused from a captured diff; 42 flake-loops + repeated full-suite runs now deterministic.)
- Three bugs found by real execution of the new layer-3 path: claude-semantic-visibility.sh --help failed pre-build; providers-semantic.sh discarded the driver's JSON evidence to /dev/null; the judge base URL was not normalized (a trailing /v1 doubled to /v1/v1/chat/completions).
- Final-review hardening: cmd\_sync/cmd\_sync\_multi heal a stale/outdated cma\_run\_provider wrapper once per sync (the byte-idempotence change had removed the per-shell self-heal, so a pre-Phase-2 gate-less wrapper could still launch); the layer-4 engagement classifier now requires the namespaced superpowers:<name> form (a bare skill word could false-verify); and verify --deep treats a layer-4 verifier crash as an honest SKIP rather than a clean verify.

## Notes

- The default providers/judge.env.template judge is DeepSeek. For strongest results, configure providers/judge.env with a judge from a **different model family** than the provider under test — 2024-2026 research (e.g. arXiv:2508.06709) shows same-family judges systematically favor their own family's outputs, including validating wrong answers. A different-family default, an xAI overrides.json base-url entry, and the submodule boundary-contract constitution id are tracked for a follow-up.

## v1.11.2 — 2026-07-03 — Token-limit guard + re-enabled provider proxies + Poe tool cap

Ran every provider alias through REAL Claude Code executing / using-superpowers (the user's reproducing prompt), in CLI mode with scrubbed env, and root-caused every genuine failure. Three

real toolkit bugs fixed (below); all remaining non-passes are account-side (insufficient funds, rejected/absent keys, or a key with no chat entitlement) — not toolkit defects.

## Fixed

- **Input token-limit 400** — `scripts/lib.sh` (`cma_run_provider`) now exports `CLAUDE_CODE_AUTO_COMPACT_WINDOW="$CMA_PROVIDER_CONTEXT_LIMIT"` before the transport branch, so Claude Code knows each provider's real input window and auto-compacts (at window - 13000) before a request overshoots it. Fixes the user-reported 400 ... exceeded model token limit: 262144 (requested: 311786) on `kimi-for-coding` and any provider whose window is smaller than the ~1M Claude Code assumes for Anthropic's endpoint. Fully dynamic/parametrized: the value is the per-model `limit.context` from the `models.dev` catalog, persisted in each provider `.env`. Guarded by `[[ -n ... ]]`; applies to **both** transports. (`CLAUDE_CODE_MAX_OUTPUT_TOKENS` only ever capped **output** — it could not fix an **input** overflow.) Verified live: `kimi's /using-superpowers` now compacts and succeeds.
- **All provider proxies were silently disabled** — `cma_run_provider` resolved compatibility proxies via `$LIB_DIR/proxy/...`, but `LIB_DIR` is a repo-only variable that does not exist in the self-contained alias file, so it expanded empty and **no proxy ever started** (Poe, etc.). Now resolves against the installed location `${SHARED_DIR:-$HOME/.claude-shared}/proxy` (where `install.sh` copies `scripts/proxy/*.py`). This is why Poe returned 400 Invalid 'tools': Field required — its request never went through the proxy that injects the required parameters field.
- **Poe tool-count limit** — Poe rejects requests carrying more than ~216 tool definitions with the same misleading 400 Invalid 'tools': Field required (verified count-based: 215 accepted, 220 rejected, independent of payload size). On this host Claude Code's large MCP-plugin load emits 400+ tools. `scripts/proxy/poe_proxy.py` now caps the tool list to `POE_MAX_TOOLS` (default 200), dropping only overflow `mcp_...` tools so **every** built-in Claude Code tool is preserved. Parametrized via the `POE_MAX_TOOLS` env var. Verified live: Poe's `/using-superpowers` now returns a successful result.

## Changed

- **cma\_ensure\_alias\_file migration** — added `CLAUDE_CODE_AUTO_COMPACT_WINDOW` and `_cma_proxy_dir` as regeneration markers, so an already-installed `cma_run_provider` predating either fix is transparently regenerated on the next alias-file touch.

- **scripts/tests/verify\_claude\_live.sh** — reclassify\_fail now maps a direct 400 whose body says the model is unknown/invalid to **BADKEY** (an account that can list models but not invoke them — e.g. inference here), while a 400 with no model-rejection marker (a real launch-layer defect) stays FAIL and is never masked.
- Corrected misleading comments that claimed the output-token cap fixed the input token-limit error; documented the two guards as independent halves.

## Added

- **scripts/tests/test\_poe\_proxy.sh** — hermetic tests for the Poe proxy: parameters injection, the tool-count cap, built-in-tool preservation, the POE\_MAX\_TOOLS override, and end-to-end fix\_request.
- **scripts/tests/test\_providers.sh** — hermetic regression tests: the emitted cma\_run\_provider exports the auto-compact window from CMA\_PROVIDER\_CONTEXT\_LIMIT only when non-empty; the migration regenerates an outdated wrapper lacking the guard while preserving surrounding alias lines.
- **scripts/tests/proof/claude-live-superpowers-cli.txt** — evidence matrix: every provider alias launched through REAL Claude Code running /using-superpowers, classified PASS / FUNDS / BADKEY / NOKEY / FAIL.

## v1.11.1 — 2026-07-03 — Live per-alias Claude Code verification (CLI + TUI) + provider fixes

Investigated the reported “most provider aliases fail with an API error.” Root cause was NOT a broad implementation defect: launched every alias through real Claude Code (scrubbed env) and probed each provider API directly. 9 aliases pass end-to-end; 2 had genuine model-config drift (fixed below); the remaining failures are all **account-side** (insufficient funds, invalid/expired keys, missing keys, or a plan with no chat model) — not toolkit bugs.

## Added

- **scripts/tests/verify\_claude\_live.sh** — end-to-end live verification of every provider alias through REAL Claude Code in BOTH modes: **CLI** (-p ... --output-format json, authoritative) and **TUI** (driven under a PTY). Each launch runs in a scrubbed env; TUI runs from a throwaway temp cwd so it can never resume a real conversation (the cross-alias .claude.json sync would otherwise auto-resume one). Outcomes are classified



**PASS / FUNDS / BADKEY / NOKEY / FAIL** so account problems are never miscounted as toolkit bugs; on a launch FAIL it probes the provider API directly to recover the true cause (e.g. a ccr hang on an upstream 401/429 → BADKEY/FUNDS, not FAIL).

- **scripts/tests/lib/pty\_drive.py** — pexpect PTY driver for the interactive Ink TUI (boot, accept any trust prompt, type prompt, capture transcript, quit).
- **scripts/tests/lib/classify\_live.py** — shared transcript classifier.

## Fixed

- **huggingface** — strong/fast pinned to non-reasoning coder models (Qwen/Qwen3-Coder-480B-A35B-Instruct / Qwen/Qwen3-Coder-30B-A3B-Instruct). The previous models emitted their answer as hidden reasoning\_content, returning empty content at low max\_tokens and reading as broken. **Verified green** in CLI + TUI.
- **kilo** — strong/fast pinned to verified free-tier models (nvidia/nemotron-3-super-120b-a12b:free / nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free). The old x-ai/grok-build-0.1 is a **paid** model this key can't access (401 → >100s stall) and the old fast baidu/cobuddy:free is retired (404). **Verified green** in CLI + TUI.
- **inference** — base URL corrected https://inference.net/v1 → https://api.inference.net/v1 (old host 301-redirects). NOTE: this key's plan currently exposes no general chat model, so the alias still errors 400 until a chat-capable plan/key is supplied — documented, not silently "fixed."
- **novita-ai** — defensive swap off the retired fast model sao10K/L3-8B-stheno-v3.2 (404).
- **claude-providers.sh (present\_key\_vars)** — skip declared-but-empty key vars so an empty key (e.g. SARVAM\_API\_KEY=) no longer spawns a broken alias that only errors at launch.

## Verified (live on host)

- **9 aliases PASS** end-to-end in CLI (and TUI smoke): chutes, huggingface, kilo, kimi-for-coding, nvidia, opencode, poe, siliconflow, zai-coding-plan.
- Non-PASS are **account-side, not toolkit bugs**: FUNDS — deepseek, fireworks-ai, novita-ai, openrouter, upstage, xiaomi, zhipuai; BADKEY — github-models, tencent-tokenhub; NOKEY — sarvam; plan-limited — inference.
- Sandbox providers suite **113/113**; shellcheck clean; python files compile.

# v1.10.8 — 2026-07-01 — noclobber-safe router-provider config write

## Fixed

- **Every router-transport provider alias broke under set -o noclobber.** `cma_run_provider` runs in the user's interactive shell; when that shell has `noclobber` set, the router-config rewrite `jq ... "$cfg" > "$tmp"` failed with *"cannot overwrite existing file"* (the just-created `mktemp` target already exists), silently dropping the update so `claude-code-router` launched with a stale/empty config → API errors. Switched the write to the `force-clobber` operator `>|`. (`lib.sh`)
- **Existing installs self-heal:** added a regeneration trigger so `cma_ensure_alias_file` rewrites an outdated `cma_run_provider` that lacks the `>|` fix (plain function-body changes previously did not re-trigger on install).

## Added (tests)

- `test_providers.sh`: regression asserting the emitted `cma_run_provider` uses `>|` (not a bare `>`) for the router-config write, plus a functional proof that `>` is blocked by `noclobber` while `>|` succeeds.

## Verified

- Suite **18/18 green**; shellcheck 0. Root cause reproduced (`set -C; echo > "$(mktemp)"` → "cannot overwrite"), fix deployed + confirmed on the host (deployed alias write line = `>| "$tmp"`).

## Note (not a toolkit bug)

- A 46-alias provider sweep shows native providers reach their APIs and return **account-side** errors (e.g. 402 Insufficient Balance) — provider billing/key state, not a toolkit fault. ## v1.10.7 — 2026-06-29 — Shared-items drift guard + audit closure

## Added (tests)

- **test\_unify.sh** — a drift guard asserting `claude-unify.sh`'s `SHARED_ITEMS` equals `lib.sh`'s `CMA_SHARED_ITEMS` (used by `claude-add-account`) minus the intentional `CLAUDE.md` special-case (`unify` promotes it via `sync_claude_md`). Catches the documented hazard of adding a shared item to one list only.
- Cleaned a pre-existing SC2319 lint in `test_unify.sh`.

## **Audited — no code changes (3 parallel investigators; every finding independently checked)**

- **Unify + rollback engine: clean** — enabledPlugins union, history.jsonl dedup, settings.json merge (malformed-input safe), plugin-manifest path rewrite, and rollback all verified correct across 170+ assertions + edge cases.
- **OpenCode sync (opencode\_sync.py): clean** — idempotent + additive verified; two reported “bugs” were refuted against the code: the setdefault “can’t update existing keys” IS the documented never-clobber design, and the --enable-all secret nit is not a security issue (an unresolved secret can’t leak) and matches “enable everything” semantics.
- **Runtime sync + add-account: clean** — the SHARED\_ITEMS “drift” is the intentional CLAUDE.md special-case (now locked by the guard above); sync-state private-key isolation, corrupt-input handling, and add-account idempotency verified.

## **Verified**

- Suite **18/18 green**; shellcheck 0.

## **v1.10.6 — 2026-06-29 — Committed credential-leak regression test**

### **Added (tests)**

- **test\_lib.sh** — a committed security regression for cma\_merge\_claude\_json: two accounts with distinct userID/oauthAccount and disjoint projects are merged; asserts each account keeps its OWN private auth keys (no cross-account leak in either direction) and the projects subtree is unioned both ways. The function previously had only indirect coverage (via the full unify workflow); this locks the property an audit verified by hand this session.

## **Verified**

- Suite **18/18 green**; shellcheck 0.

# v1.10.5 — 2026-06-29 — Provider ‘null’ field normalization + coverage

## Fixed

- **A missing JSON field could write CMA\_PROVIDER\_MODEL='null' (and TRANSPORT, alias name) into a provider env file,** launching the provider with a bogus model.  
cma\_provider\_write\_env normalized base/fast/context/max “null”→empty but missed model and transport; claude-providers multi-sync also extracted strong\_model/transport/alias\_name with bare jq -r (no // empty, unlike the already-correct context\_limit/max\_output). Fixed at both the source (// empty on every extraction) and the choke point (normalize model+transport in cma\_provider\_write\_env). Reproduced (='null'), confirmed fixed (='').

## Added (tests)

- **test\_providers.sh** — regression asserting a null model/transport/base/etc. is normalized to empty; no field ever contains the literal 'null'.
- **test\_session.sh** — EXECUTION tests for the hint subcommand (run on every bare launch, previously only string-matched) and for cma\_project\_root’s git-toplevel + symlink (pwd -P) branches.

## Audited — no change needed (independently verified, not taken on trust)

- cma\_merge\_claude\_json: NO cross-account credential leak; projects unioned; corrupt input skipped gracefully (verified with crafted 2-account inputs).
- BSD/macOS portability: no unguarded GNU-isms (the 3 readlink -f hits are comments; stat -f/-c branch + cma\_realpath guards present).
- jq robustness: the @tsv sync paths render null as empty (safe); two reported 2>&1 “error-leak” findings were FALSE — there is no 2>&1 on those lines.

## Verified

- Suite **18/18 green**; shellcheck 0.

## v1.10.4 — 2026-06-29 — set -e/pipefail abort fixes + hardened test coverage

### Fixed

- **claude-providers list / remove aborted on a provider with no alias line.** Under set -euo pipefail, the alias-name probe `grep ... | sed | head -1` returns 1 (no match) when a provider's .env exists but its alias line is absent (manual edit / partial setup); pipefail propagated the failure and set -e killed the subshell (list) or the function before `rm -f` (remove). Guarded both with `|| alias=""`. (claude-providers.sh)
- **cma\_ensure\_alias\_file aborted on an alias file lacking export CLAUDE\_BIN=.** The CLAUDE\_BIN-migration probe `grep -m1 '^export CLAUDE_BIN=' ...` returned 1 on an older/hand-edited alias file and aborted the function mid-run under set -e. Guarded with `|| _cur_cb=""`. (lib.sh)

### Changed (tests)

- **test\_providers.sh** — replaced the AT-RISK fixed-window `grep -A40 '^cma_run()'` assertions (the push marker had drifted to within 9 lines of the window edge, the same brittleness that already broke -A30 once) with full-body awk extraction; added EXECUTION regressions that run the real claude-providers list/remove against an alias-less provider and assert no abort.
- **test\_coverage.sh** — added a regression that EXECUTES `cma_ensure_alias_file` against an alias file with no `export CLAUDE_BIN=` line and asserts it completes.
- **test\_session.sh** — added EXECUTION tests for the hint subcommand (run on every bare launch, previously only string-matched): exits 0, writes only to stderr, names the snake\_case project, handles an empty label.

### Verified

- Suite **18/18 green**; shellcheck 0. All three aborts reproduced (RED) and confirmed fixed (GREEN); the providers fix proven RED on a guard-stripped copy. Found via 3 parallel investigator subagents, each finding independently reproduced before fixing.

## v1.10.3 — 2026-06-29 — Execution-level wrapper test coverage

### Added

- **test\_wrapper\_exec.sh** — the first hermetic test that actually *executes* the generated `cma_run` wrapper (every other suite only string-matches its emitted body, so a runtime bug — a set `-e` abort, a dropped unset, wrong call order — could ship past a green suite). It drives `cma_run` with a stub `CLAUDE_BIN` env-recorder plus stub `claude-session/claude-sync-state`, then asserts `RUNTIME` guarantees: provider-env isolation (a leaked `ANTHROPIC_BASE_URL/AUTH_TOKEN/MODEL` is genuinely cleared *before* claude runs), session flags reach claude on a bare launch, `sync-state` pull fires before launch and push after, explicit args pass through verbatim with no session-flag injection, plus a non-vacuity guard proving the stub claude really executed. Proven **RED** on a dropped unset, **GREEN** on the real wrapper.

### Verified

- Suite **18/18 green**; shellcheck 0.

## v1.10.2 — 2026-06-29 — Self-healing rc source lines + strict rc tests

### Fixed

- **Dangling source `".../aliases.sh"` lines in rc files.** A transient or moved alias-file path could leave a source line in `~/.bashrc/~/.zshrc` pointing at a deleted file, so every new login shell printed `-bash: .../aliases.sh: No such file or directory`. `cma_ensure_alias_file` now **prunes** any rc source/. line whose `aliases.sh` target no longer exists (self-heal on the next install), and recognizes an existing source line across `./source` and `$HOME/~`/absolute forms, so re-installs never accumulate duplicate source lines.

### Added

- **test\_rc\_sourcing.sh** (10 strict assertions) — reproduces the bug class the hermetic suite missed (it sandboxes `$HOME` and never inspected or *sourced* the rc files): prune drops dangling / keeps valid + comments + unrelated lines, ensure self-heals, **a fresh shell sources the rc with NO error** (the reported symptom), idempotent (exactly one source line after 3 calls), and cross-

form dedup. Proven RED on the old behavior, GREEN on the fix.

## Verified

- Suite **17/17 green**; shellcheck 0.

## v1.10.1 — 2026-06-29 — Robust cma\_run wrapper assertions

### Fixed

- **test\_claude.sh** used a fixed `grep -A30 window` to scan the `cma_run` body and silently missed the sync-state push marker once the body grew with the v1.10.0 `apply-color` calls (push slipped past line 30) — failing the suite against a v1.10.0-installed alias file even though the wrapper itself was correct. It now extracts the full function body (awk header → closing brace), robust to future growth.

## Verified

- Suite **16/16 green** against the v1.10.0 wrapper; shellcheck 0.

## v1.10.0 — 2026-06-29 — Auto-applied per-alias session color + coverage/wiring

The per-alias session color is now **auto-applied** (it was only a hint in v1.9.x), plus self-healing for a stale `CLAUDE_BIN` and several closed test-coverage gaps.

### Added

- **Auto-applied per-alias session color.** Each bare alias launch now writes the alias's color into the session as an agent-color record — the exact record Claude Code's `/color` writes — via the new `claude-session apply-color`, called by `cma_run/cma_run_provider` (before launch to colour a resumed session; after exit to colour a freshly-created one). Deterministic `md5(label) mod 8` over red/blue/green/yellow/purple/orange/pink/cyan: each alias gets a stable, distinct colour, and switching the same session between aliases re-colours it. Verified **LIVE** on claude 2.1.195 — written, idempotent, persists across `--resume`. (Prompt-bar rendering must be

confirmed visually: `/color` is TUI-only and `claude -p '/color x'` is a no-op, so record injection is the only non-interactive mechanism. See [docs/SESSION\\_COLOR.md](#).)

- Test coverage: `test_install.sh` (executes `install.sh` in a sandbox — symlinks, alias file, idempotency), `test_verify_scripts.sh` (`model_verify.py`
  - `providers-verify.sh`), `test_session` apply-color tests (incl. the `set -e` regression), `test_coverage` B7 (CLAUDE\_BIN resolver), B8 (CLAUDE\_BIN migration), B9 (apply-color wired into both wrappers). `run-proof.sh` now also runs the previously-orphaned `verify_aliases_live.sh`.

## Fixed

- **Stale CLAUDE\_BIN self-heals.** Existing installs whose alias file pointed CLAUDE\_BIN at a non-existent path (e.g. `~/.local/bin/claude` where `npm` put `claude` in `~/.npm-global/bin` — the `amber.local` case) now rewrite it to a resolved, executable `claude` on the next `install/ensure`.
- A `set -e/pipefail` bug where `apply-color` aborted before writing on a session that had no existing `agent-color` record.

## Verified

- Suite **16/16 ALL GREEN; shellcheck 0**. Color injection proven **LIVE** on real `claude 2.1.195` (`write / idempotent / persist-across-- resume / recolour-on-alias-switch`).

## v1.9.2 — 2026-06-29 — Hermetic CLAUDE\_BIN resolver test

### Fixed

- `test_coverage.sh` B7 “**fallback when nowhere**” was not **hermetic** and failed on hosts that have a real `/usr/local/bin/claude` (caught live on `thinker.local`): the resolver *correctly* returns the system `claude` there, but the test wrongly assumed “`claude nowhere`” was achievable under a sandboxed `HOME/PATH` (it can’t mask absolute system paths). Dropped that one assertion; the load-bearing discovery cases (explicit `CLAUDE_BIN`, `~/.npm-global/bin` discovery) stay covered. Runtime behavior unchanged.

### Verified

- Suite **14/14 green on all five hosts** (this host, `mistborn`, `thinker`, `amber`, `nezha`); `shellcheck 0`.



## v1.9.1 — 2026-06-29 — CLAUDE\_BIN resolves across per-host install locations

A patch found during the live multi-host rollout of v1.9.0.

### Fixed

- **Alias launches failed where claude was installed outside ~/.local/bin.** npm i -g @anthropic-ai/claude-code lands in different prefixes per host (~/.npm-global/bin, Homebrew, ~/.local/bin); the toolkit's fixed CLAUDE\_BIN default mis-pointed on those hosts, so every claudeN/provider launch failed "No such file or directory" (amber.local needed a manual symlink to work). cma\_resolve\_claude\_bin now prefers an explicit CLAUDE\_BIN, then \$PATH, then the known locations (~/.local/bin, ~/.npm-global/bin, /opt/homebrew/bin, /usr/local/bin), with a ~/.local/bin fallback.

### Verified

- Suite **14/14 green; shellcheck 0**. test\_coverage.sh B7 covers explicit / npm-global / fallback resolution. v1.9.0's auto-session naming confirmed installed + **live-validated** (create-named + legacy-rename) on all five hosts: this host, mistborn, thinker, amber, nezha.

## v1.9.0 — 2026-06-29 — Per-project auto-sessions that actually work live + zero-coverage tests

A minor release that makes the v1.8.0 "auto session-per-project" feature do what it promised. As shipped, opening any alias gave an **unnamed** session; three root causes — all reproduced and fixed against the real claude 2.1.195 binary, then proven **LIVE end-to-end** — are corrected here, plus test coverage for two zero-coverage utilities and a documentation refresh.

### Fixed

- **Per-project auto-session naming never actually named the session — now proven LIVE.** Three independent root causes:
  - **Legacy/unnamed sessions were never renamed.** The launcher only --resume'd an existing session and never passed --name, so a session created by an older wrapper or

by plain `claude` stayed unnamed forever. Fix: always pass `--name` on resume too. Proven live — `claude --resume <id> --name <x>` renames a previously-unnamed session (custom-title `<NONE>` → `<x>`), contradicting the docs but confirmed empirically.

- **The session-existence check used a run-collapsing slug.** It collapsed runs of non-alnum to one - (`s/[^A-Za-z0-9]+/-/g`), but `claude` slugs **per char**, so paths with consecutive non-alnum segments (hidden dirs, `/tmp/.private`, `__pycache__`) false-negated the lookup and **re-created** instead of resuming. Fix: per-char slug (`s/[^A-Za-z0-9]/-/g`), matching the real on-disk dir names.
- **cma\_run self-heal regenerated only on a missing unset ANTHROPIC\_ marker.** Wrappers predating auto-session carried that marker but not the `claude-session` one, so they never regained the integration. Fix: regenerate when **either** marker is missing.

## Added

- **docs/SESSION\_COLOR.md** — resolves the previously dangling reference, documenting per-project auto-session naming and the honest `/color` limitation: in `claude 2.1.195`, `/color` is **TUI-only** (no CLI flag, no `settings.json` key, no env var — verified against the binary and the docs), so the toolkit can only print a deterministic per-alias hint, never auto-apply it.
- **test\_toon.sh (9 assertions) and test\_bootstrap.sh (39 assertions)** — both utilities previously had **zero** coverage. `toon` (hermetic, SKIP-if-no-node): `toon.mjs` encode/decode round-trip, the `toon_encode.py` `python→mjs` chain, and non-zero exit on invalid JSON. `bootstrap` (hermetic): `claude-bootstrap --count 2 --yes` in a sandbox `$HOME` asserting account dirs, shared symlinks, private-file isolation, alias lines, and the documented refuse-to-clobber re-run behavior.
- **test\_coverage.sh B6** asserts the emitted `cma_run / cma_run_provider` bodies actually carry the auto-session integration (bare-launch guard, `claude-session` flags, `eval set -- apply`, color hint) — the session script's own unit tests can't see the wrapper — plus a **self-heal regression**: a stale `cma_run` missing the `claude-session` marker is regenerated (exactly one `cma_run()`, provider-env isolation retained, aliases preserved). `test_session.sh` updated for `--name-on-resume` and a per-char-slug regression (a `/.cfg/` path must resume, not re-create).
- **Docs refresh.** README rewritten as a project landing page; `scripts/README` refreshed with the full current script inventory; the long-form guide gained “Per-project auto-session & per-alias color” and “TOON utility” sections; the `/color` notes pinned to the verified `claude 2.1.195` (superseding older 2.1.178 references).

## Verified

- Full suite **14/14 ALL GREEN**; **shellcheck 0**. The session fix proven **LIVE end-to-end** on the real `claude 2.1.195` binary — fresh create, resume, and legacy-unnamed rename all confirmed. New tests are non-vacuous (concrete expected values / negative controls). Installed live on this host and validated.

## v1.8.1 — 2026-06-29 — Merge-engine correctness + portability hardening

A patch release: an adversarial correctness audit of `claude-unify`'s merge engine plus a BSD/GNU portability pass over the test + proof tooling. All fixes/hardening — **no new features**.

Housekeeping: a divergent mirror lineage that re-created `v1.7.11` (`1e975e5`) was merged back into main resolved to **OURS** (local already carries `v1.7.11` → `v1.8.0` and later fixes that supersede it), leaving a tree byte-identical to HEAD so all four mirrors converge on one lineage; the containers submodule was fast-forwarded to latest main (`71d3256` → `67ed35a`).

## Fixed

- **history.jsonl merge fused records across a source missing its trailing newline.** `merge_history_jsonl` cat'd sources into a temp first, gluing one file's last line onto the next file's first line → two entries collapsed into one invalid-JSON line. Fix: feed files straight to `awk` (fresh record per file). Regression **R1** (RED before, GREEN after).
- **enabledPlugins union dropped "any true".** The `jq` used `+/` (rightmost-wins), so a plugin enabled in an earlier account but false in the lexically-last account ended up disabled for everyone — contradicting the documented "any true survives" guarantee. Fix: `OR-of-true` reduce over every account. Regression **R2**.
- **A single malformed settings.json aborted the whole unify — and naive guarding then risked silent config loss.** The multi-file `jq -s` ran unguarded under `set -e` (`settings` is item 15 of 16), halting mid-run. Merely skipping the merge was worse: `link_to_shared` still replaced each valid account's real `settings.json` with a symlink to a never-written target (a dangling link → silent loss, exit 0). Final fix (hardened after adversarial review): validate each file with `jq empty` and merge only the valid ones (a malformed sibling is excluded, not fatal), and `link_to_shared` refuses to create a link when the shared target is absent. Regression **R3** (asserts the valid account's settings stay readable, not just that unify exits 0).

- **Directory-merge conflicts were resolved by lexical account name, not recency.** `merge_dir_into_shared`'s second pass overlaid only `ACCOUNTS[-1]` (alphabetically-last) while claiming to bias toward the “most recently active” account, so a stale account sorting last could clobber fresher memory/\*.md. Fix: overlay every account with `rsync -au` so the newest-mtime file wins each conflict, independent of name/order. Regression **R4**.
- **Rollback left dangling symlinks.** Unify symlinks every shared item into each account; for an item an account never had there is no `.preunify` backup, so rollback's restore loop never visited it and the symlink dangled once `SHARED_DIR` moved aside. Fix: after restoring backups, remove any leftover symlink whose target points into `SHARED_DIR` (skipping the shared store itself). Regression **R5**.
- **Rollback restored a non-deterministic backup.** `find -print0` was unsorted, so when a path had several `.preunify.*` backups an arbitrary one was restored. Fix: `sort -z` (timestamps are `YYYYMMDDHHMMSS` = lexical-chronological) so the oldest — the true pre-unify original — wins. Regression **R6**.
- **test\_unify.sh B2 was a vacuous PASS.** It called `cma_realpath` (a `lib.sh` function) without sourcing `lib.sh`, so the call errored to empty and the assertion compared `"" == ""` — the symlink target was never verified. Fix: `source lib.sh + set +e` (matching every sibling test that uses `lib` functions directly). Now prints the real resolved `SHARED_DIR/plugins/cache` path.
- **Portability: 3 GNU-only constructs broke the test/proof tooling on macOS** (the shipped runtime toolkit was already clean). `readlink -f` (no `-f` on BSD) in `assert_symlink_to/` `test_unify.sh` returned empty → spurious symlink pass/fail, fixed with a self-contained `_assert_realpath` in `assert.sh + cma_realpath` in `test_unify.sh`; `sed -E 's/\x1b.../'` (\xNN is GNU-sed-only) in `run-proof.sh/verify_opencode_live.sh` left ANSI in, skewing `grep -c` counts, fixed by building the ESC byte via `printf '\033'`; unguarded timeout (GNU coreutils) in `verify_opencode_live.sh`, fixed by resolving `timeout/gtimeout` once and degrading if absent.
- **De-vendored node\_modules.** `node_modules/@toon-format/toon` was committed by an accidental “Auto-commit” yet load-bearing (`scripts/toon.mjs` imports the bare specifier; `toon_encode.py` shells out to it) with nothing ever running `npm install`. Removed from the tree (`git rm --cached + gitignore / node_modules/`). Proven by fresh-clone simulation: `ERR_MODULE_NOT_FOUND` before, encodes correctly after.
- **mktemp portability.** Standardized every bare `mktemp [-d]` to the templated `mktemp [-d] "${TMPDIR:-/tmp}/cma.XXXXXX"` form `CLAUDE.md` prescribes (BSD `mktemp` requires a template; only GNU tolerates a bare call) across `lib.sh`, `claude-unify/providers/opencode-sync/session/install`, and the test harness (the sandbox keeps its `cma-test.` prefix for the cleanup safety check).

## Added

- **B5 token-limit guard coverage** (test\_coverage.sh). The v1.8.0 context\_limit/max\_output path (cma\_provider\_write\_env → CMA\_PROVIDER\_CONTEXT\_LIMIT/CMA\_PROVIDER\_MAX\_OUTPUT → cma\_run\_provider exporting CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS) shipped with **zero** tests — the only v1.8.0 fix lacking one. 4 cases / 6 concrete-value assertions: round-trip (262144/32768), null→empty normalization, 7-arg back-compat, and the emitted wrapper carrying the export.
- **npm install step in install.sh** (soft — warns, never hard-fails, when npm is absent; core unify/add-account needs no Node), so a fresh clone gets @toon-format/toon without a vendored tree. curl-install.sh inherits it via delegation.
- **+16 regression assertions** — 6 in test\_coverage.sh (B5) and 10 in test\_unify.sh (R1-R6 above), each written RED-before / GREEN-after.
- **Documented two deliberate merge/sync trade-offs** in-code so they are explicit rather than silent: cma\_merge\_claude\_json replaces (not element-unions) array values; claude-sync-state pull/push is last-writer-wins (no portable mutex is worth its stale-lock failure modes for a per-launch hook).

## Verified

- Full suite **12/12 ALL GREEN; shellcheck 0**; all .py compile under python3 -W error. Each bugfix proven **RED-before / GREEN-after**; the de-vendor proven via fresh-clone simulation (node scripts/toon.mjs + toon\_encode.py); ESC-strip verified functionally; the post-merge tree confirmed byte-identical to HEAD. Installed live on this host and validated against all existing aliases (3 native + 44 provider) + claude-list-accounts.

## v1.8.0 — 2026-06-29 — Alias isolation + token-limit guard + per-project auto-sessions

A systematic-debugging pass fixing three reported issues plus a new session-per-project feature. Every root cause was reproduced and the fix proven with physical evidence before shipping.

## Fixed

- **CRITICAL — aliases cross-contaminated API endpoints across sessions.** cma\_run\_provider exports ANTHROPIC\_BASE\_URL/ AUTH\_TOKEN/MODEL/ SMALL\_FAST\_MODEL into the interactive shell, and native cma\_run did **not** clear them — so running a provider alias

(e.g. xiaomi) and then a native alias (claude1) in the same shell made the native one inherit the provider's endpoint (api.xiaomimimo.com). cma\_run now unsets those four vars before launch. Proven live: after a leaked xiaomi env, native launch shows ANTHROPIC\_BASE\_URL=<unset>. Existing installs auto-regenerate the wrapper (migration keyed on the new unset ANTHROPIC\_marker).

- **Token-limit 400 (“exceeded model token limit: 262144”).** The models.dev catalog's per-model limit.context / limit.output were read for ranking but never emitted, so Claude Code overshot a provider's real context window. providers\_resolve.py now emits context\_limit + max\_output; providers\_generate.py and cma\_provider\_write\_env write CMA\_PROVIDER\_CONTEXT\_LIMIT / CMA\_PROVIDER\_MAX\_OUTPUT into each .env; and cma\_run\_provider exports CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS from it. Proven: kimi-for-coding now resolves context\_limit=262144 max\_output=32768 from the live catalog.
- **“workspace has not been trusted” warning on launch.** Confirmed NOT a merge bug (trust propagates across accounts correctly); the warned project was simply never trusted under any account. The launch wrapper now writes projects[<root>].hasTrustDialogAccepted=true for the launching project via the new claude-session helper.

## Added

- **Auto session-per-project (claude-session.sh).** Every bare alias launch (native or provider) now resumes — or, the first time, creates — one long-lived Claude session per project root: stable --session-id (UUID derived from the git-root path), --name set to the root dir basename in lowercase snake\_case (Android 15 → android\_15). Explicit args/flags are always respected verbatim. Verified against the real claude CLI: --session-id creates, --resume resumes.
- **Per-alias color hint.** A deterministic alias→color mapping over Claude Code's real palette (red blue green yellow purple orange pink cyan); printed as a /color <x> tip on launch. (Investigated thoroughly: /color is a TUI-only command with no CLI flag / settings key / writable persistence, so it cannot be auto-applied — the toolkit suggests it rather than faking it.)
- **test\_session.sh** — 27 hermetic assertions for name/id/color/flags/trust/ git-root behavior. **run-all.sh is now 12 files / 60 assertions, ALL GREEN.**

## Verified

- Full suite **12/12 ALL GREEN; shellcheck 0**; all .py compile under python3 -W error. All four items proven end-to-end against the live catalog and the emitted alias file.

# v1.7.12 — 2026-06-28 — One-line curl installer

## Added

- **curl-install.sh** — one-line bootstrap installer:

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/claude-toolkit/main/scripts/curl-install.sh | bash
```

Detects platform (Linux/macOS) and shell, auto-installs missing hard dependencies (jq, rsync, awk) via the system package manager (apt/dnf/apk/pacman/brew), clones (or pulls if already present) the repo with all submodules recursively to ~/claude-toolkit, runs install.sh, and prints next-steps. Idempotent; re-runnable. Install dir overridable via CLAUDE\_TOOLKIT\_DIR env var.

- **README.md** — curl one-liner added at the top of the Install section.
- **test\_curl\_install.sh** — 22 hermetic tests covering syntax, permissions, URL correctness, submodule cloning, idempotency, platform detection, dependency checks, error handling, and next-steps output.

## Verified

- bash -n + shellcheck 0 on curl-install.sh and test\_curl\_install.sh.
- run-all.sh **11/11 ALL GREEN** (was 10; +test\_curl\_install.sh).

# v1.7.11 — 2026-06-28 — Round-4: coverage-gap regression tests, toon recursion guard, arg validation

Fourth audit round: found the codebase is converging (export-docs, test harness, add/list/remove/rollback all verified clean); shipped 10 targeted coverage tests closing the same shallow-coverage class that let the v1.7.10 enable-plugins bug ship; plus 2 LOW code findings.

## Fixed

- **toon\_encode.py fallback\_encode**: unbounded recursion on deeply nested JSON. A crafted input with >1000 levels would blow

Python's default stack and exit with an unhandled traceback instead of encoding. Added a `_depth` guard; beyond 64 levels emits compact JSON as a safe fallback.

- **toon.mjs encode-file / decode-file**: running `toon.mjs encode-file` with no argument produced a confusing `TypeError` from Node's `fs.readFileSync`. Now prints  
Error: encode-file requires a filename argument + exit 1.

## Added — coverage-gap regression tests (the same class that let enable-plugins)

ship) - **B9 (HIGH) — cma\_ensure\_alias\_file migration path** (`test_coverage.sh`): builds a realistic old `cma_run_provider()` body lacking `claude-sync-state`, calls `cma_ensure_alias_file`, asserts the body is migrated, the following alias `claude1=` survives, and `cma_run_provider()` appears exactly once. - **B3 (HIGH) — \_cma\_q bash quoting in cma\_provider\_write\_env** (`test_coverage.sh`): sources a `.env` with a model name containing a literal single quote and asserts it round-trips intact; also asserts an injection payload does NOT execute on source (mirrors the already-tested Python `q()`). - **B1 (HIGH) — absorb\_default\_plugins** (`test_unify.sh`): creates a real plugin file under `$HOME/.claude/plugins/cache/` before unify; asserts it lands in `$SHARED_DIR/plugins/cache/`. - **B2 (HIGH) — link\_default\_plugin\_subdirs** (`test_unify.sh`): asserts `$DEFAULT_DIR/plugins/cache` becomes a symlink into `$SHARED_DIR/plugins/cache` after unify, and that re-running unify doesn't create a second backup. - **B4 (MEDIUM-HIGH) — sync\_claude\_md seed branches** (`test_unify.sh`): branch (b) seeds `$DEFAULT_DIR/CLAUDE.md` and asserts it wins; branch (c) removes it and gives an account a `CLAUDE.md`, asserts that one wins.

## Verified

- `run-all.sh` **10/10 ALL GREEN** (coverage now  $39+10=49$  assertions; unify now  $43+7=50$ ); **shellcheck 0**; all `.py` compile under `python3 -W error`; `node --check toon.mjs` clean; `toon_encode 500-level-nest` no longer crashes; `toon.mjs missing-arg` gives clean error + exit 1.

## v1.7.10 — 2026-06-28 — Round-3 audit: enable-plugins bug fix, path-traversal guards, proxy robustness

Third audit round (deep dive on the less-covered surface: `opencode_sync`, `claude-unify` merge, the `poe` proxy, bootstrap). Fixes verified centrally.



## Fixed

- **cma\_enable\_plugins silently enabled NO plugins when given 3 or more** (lib.sh). The jq --arg index was derived as  $\$ \{ \# \text{args} [ @ ] \} / 2$ , but each iteration appends **three** elements — so for the default 4 always-on plugins it produced arg names p0,p1,p3,p4 while the jq program referenced \$p0..\$p3; \$p2 was undefined, jq failed, 2>/dev/null swallowed the error, and enabledPlugins was left empty. Replaced the derived index with a dedicated counter. Proven live: cma\_enable\_plugins a b c d now yields all four true (was empty); a  $\geq 3$ -plugin regression test was added.

## Security (defense-in-depth)

- **Path traversal via unvalidated provider id** in claude-providers.sh cmd\_show / cmd\_remove: \$id was interpolated into <dir>/\$id.env and then cat/rm -f'd without validation. Now rejected unless it matches [A-Za-z0-9.\_-] (blocks ../), matching cma\_provider\_write\_alias.
- **opcode\_sync.py \${CLAUDE\_PLUGIN\_ROOT} path traversal:** a malicious installed plugin could set an arg like \${CLAUDE\_PLUGIN\_ROOT}/../../../../tmp/evil.js and --enable-all-local would have OpenCode exec the traversed path. Expansion now lexically contains the result to the plugin dir; an escaping value is left unexpanded (fails safe).
- **cma\_write\_alias now rejects whitespace in the config dir** (lib.sh): an unquoted space silently word-split the alias into a bogus command — now a clear error instead of a broken alias.

## Robustness

- **poe\_proxy.py:** resolve\_refs gained a recursion-depth guard (a circular \$ref previously crashed the request handler with RecursionError); the success-path gzip.decompress is now guarded like the error path, so a corrupt gzip body no longer propagates.

## Verified

- run-all.sh **10/10 ALL GREEN** (coverage now 32 assertions incl. the enable-plugins + injection regressions); **shellcheck 0**; all .py compile under python3 -W error.
- cma\_enable\_plugins fix proven live (4 plugins → all true); opcode containment + id validation proven with PoCs. The model-verification / alias- write path is unchanged from v1.7.9's live-proven 137 models / 32 aliases.

## Audit (round 3) — verified clean

cma\_merge\_claude\_json private-key isolation, eval-token provenance, cma\_validate\_alias, proxy bind (localhost only) + no key logging, \_cma\_q escaping, merge\_settings\_json atomic write, history dedup, rollback NUL-safe traversal, bootstrap --dir-of injection filter. (opencode\_sync --enable-all intentionally bypasses the needs-secret guard — operator opt-in, documented.)

## v1.7.9 — 2026-06-28 — Hardening round 2: injection-safe alias writes, broadened secret redaction, docs accuracy, shellcheck 0

A second multi-agent audit + hardening pass on top of v1.7.8 (adversarial security audit + doc-accuracy audit + lint sweep, fixes verified centrally).

### Security

- **Provider id / config dir can no longer inject shell via the alias file** (lib.sh cma\_provider\_write\_alias / cma\_write\_alias). Both interpolate values into alias name="..." lines that the shell **re-parses on invocation**, and jq @tsv does not escape ". They now reject shell metacharacters (provider id restricted to [A-Za-z0-9.\_-]; config dir rejects " \$ \ ; & | < > ( ) and newline). Proven: afoo"; touch ...` payload is rejected, no command runs, the hostile alias is never written.
- **Keys-file read no longer breakable by a quote in the path** (claude-providers.sh cmd\_sync\_multi). The old bash -c "set -a; source '\$keysf'; ..." let a single quote in the keys-file path break out of the string. Replaced with an isolated subshell ( set +e; set -a +u; . "\$keysf"; set +a; eval ... ) — the same safe pattern cmd\_sync already used. Proven with a do n't/ path.
- **.env value quoting** (providers\_generate.py): q() now POSIX single-quote-escapes embedded quotes (mirrors lib.sh \_cma\_q), so a catalog value containing a quote can't inject when cma\_run\_provider sources the .env. Proven: an injection payload is neutralized to a literal.
- **xtrace secret leak** (lib.sh cma\_run\_provider): the indirect key read is now wrapped in set +x/restore so an active set -x in the user's shell can't echo the key to the terminal or a redirected log.
- **Broadened secret redaction + guard**: cma\_redact\_secrets() (verify\_opencode\_live.sh) and the committed-proof scan guard (test\_lib.sh) now also catch sk-ant-, hf\_, AIza, xoxb-/xoxp-/xoxs-, pc-, re\_, secret\_, and JWTs — regardless of JSON field name — closing the gap where arbitrary MCP env-var names

(e.g. NOTION\_API\_KEY) slipped through the original six-name allowlist.

## Fixed

- **install.sh used readlink -f** (absent on BSD/macOS) for its symlink up-to-date check — missed by the v1.7.7 sweep. Now uses `cma_realpath`; the `test_lib.sh` guard scans `install.sh` too.
- **verify\_aliases\_live.sh hardcoded one developer's account dirs**, producing false FAILs on every other host. Now discovers accounts dynamically and skips dirs that don't exist.
- Dead code / cruft: `providers_generate.py` (unused import, dead vars, `lambda→def`, a no-op `provider_id + ('' if ... else '')`); `model_verify.py` (unused import `hashlib`); `model_verify.py` docstring `--key → CMA_PROBE_KEY`.

## Docs

- Long-form doc + READMEs + CLAUDE.md corrected against the code: macOS rc-file caveat (`~/ .zshrc` only), the test table now lists all 10 suites, the full installed-command list (`+claude-providers/claude-sync-state/ claude-bootstrap`), repo-relative paths (was `~/Documents/scripts/`), a new `claude-bootstrap` section, the `CMA_PROBE_KEY` security model in §11, and a refreshed date stamp.

## Quality

- **shellcheck: 93 → 0** across all scripts. Added `.shellcheckrc` (`external-sources=true`) which resolves the sourced-file warnings properly; fixed the  `$?-after-condition` (SC2319) test idioms, SC2015/SC1090/SC1003; the one remaining reserved no-op flag carries a justified inline disable.

## Verified

- `scripts/tests/run-all.sh` **10/10 ALL GREEN**; **shellcheck 0**; every `.py` compiles under `python3 -W error`.
- Injection PoCs (provider id, config dir, `.env` value, keys-file path) all proven neutralized; broadened redaction proven against `AIza/hf_/JWT/etc`.
- Live sync `--multi`: **137 models verified, 32 aliases** across 8 providers (opencode 4, poe 33, chutes 7, huggingface 6, nvidia 30, openrouter 14, siliconflow 38, xiaomi 5), zero `CMA_PROBE_KEY/unbound errors` — identical to the v1.7.8 baseline, so the new key-read path is non-regressive end-to-end.
- 4-host byte-parity + 10/10 suite re-verified after deploy.

## v1.7.8 — 2026-06-28 — Secret hygiene (argv + committed-proof leaks), dead-code fix, coverage tests

Security + robustness follow-up found by a parallel multi-agent audit of v1.7.7. Four independent subagents fixed disjoint file sets; integration + the full suite + live multi-model verification were run centrally.

### Security

- **API key no longer passed on argv** (model\_verify.py + claude-providers.sh). cmd\_sync\_multi invoked model\_verify.py --key "\$token", placing the secret verbatim in /proc/<pid>/cmdline and ps aux output — readable by any user on a multi-user host. The key now flows via the CMA\_PROBE\_KEY environment variable (set per-command, not exported); model\_verify.py reads it from the environment and errors clearly if unset. The --key flag is removed entirely.
- **API key no longer passed to curl on argv** (verify\_aliases\_live.sh). Six live-probe calls used -H "Authorization: Bearer \$key". The header is now written to a mktemp'd, chmod 600 config file consumed via curl --config (portable on GNU + BSD curl) and removed via an EXIT/INT/TERM trap.
- **Leaked secrets purged from committed proof artifacts** (committed in 24bc379, rolled into this release): the OpenCode live verifier wrote resolved opencode debug config / mcp list output — which contained a real provider key and a DB connection-string password — verbatim into the committed proof dir. The three artifacts are redacted; the generator (verify\_opencode\_live.sh) now redacts via cma\_redact\_secrets() before writing (raw dump → .raw temp → redacted file → .raw removed). **Operator follow-up still required:** rotate the leaked key and decide on a git-history scrub — the values remain in history on all four remotes.

### Fixed

- **Unreachable code** in verify\_aliases\_live.sh: exit \$failed sat *before* the Claude-alias test function and its caller, making them dead (shellcheck SC2317). exit \$failed moved to the final statement.
- **Fragile \$? capture** in test\_list.sh: grep ...; [[ \$? -ne 0 ]] then assert\_eq 0 \$? read \$? from the wrong command. Now captures rc=\$? immediately.
- **Unquoted glob** in claude-sync-state.sh:67: "\$HOME"/\${ACCOUNT\_PREFIX}prov-\* / → "\$HOME/\${ACCOUNT\_PREFIX}"prov-\* / so only the intended \* globs.

- **SyntaxWarning: invalid escape sequence '\ ' in** providers\_resolve.py: the usage docstring's \ line-continuations are now a raw string (r""").

## Added

- **test\_coverage.sh** — 11 new hermetic tests (19 assertions) covering previously-untested lib.sh behavior: cma\_ensure\_alias\_file (fresh / idempotent / old-format migration preserving unrelated lines), cma\_can\_prompt (CMA\_NONINTERACTIVE=1 and no-tty both non-interactive), cma\_enable\_plugins (JSON shape + additive + the jq // falsy-vs-null upgrade), cma\_link\_shared\_items (every CMA\_SHARED\_ITEMS entry becomes a symlink into \$SHARED\_DIR, idempotent), and stats-cache.json newest-by-mtime selection.
- **Proof regression guard** (test\_lib.sh): scans scripts/tests/proof for provider-key prefixes and URL user:password@ creds, counting suspect lines so a failure never re-echoes a secret.

## Verified

- scripts/tests/run-all.sh — **10/10 ALL GREEN** locally (was 9; +test\_coverage.sh).
- Live multi-model verification (claude-providers.sh sync --multi, real HTTP probes with the host's real keys): **137 models verified, 32 aliases generated** across 8 providers (opencode 4, poe 33, chutes 7, huggingface 6, nvidia 30, openrouter 14, siliconflow 38, xiaomi 5). Zero CMA\_PROBE\_KEY-unset and zero unbound variable errors — the env-var key path works end-to-end. (Providers with 0 verified are external: dead/paid keys, HTTP 401/402/403, WAF blocks — not toolkit regressions.)
- The new proof secret-scan guard immediately earned its keep: on first cross-host run it flagged a **stale, pre-redaction proof dir on all three remote hosts** (3 files with literal secrets), which were then re-synced with the redacted artifacts.
- model\_verify.py / providers\_resolve.py compile clean under python3 -W error.

## v1.7.7 — 2026-06-28 — Portable realpath (BSD portability hardening), set -u edge fix, regression tests

Follow-up hardening release found by a parallel multi-agent audit of v1.7.6.

## Fixed

- **readlink -f** → **portable cma\_realpath** at three sites: `claude-unify.sh` (already\_linked\_to\_shared and merge\_settings\_json) and `claude-list-accounts.sh` (the link check). `readlink -f` is absent on older macOS and on other BSDs (FreeBSD/NetBSD); there the checks silently fail — making `claude-unify` re-link every shared item on each re-run (accumulating stale `.preunify.*` backups) and `claude-list-accounts` report linked accounts as “not linked”. **Honest scope:** modern macOS (Sequoia) and GNU coreutils DO support `readlink -f`, so on the current fleet this was a *latent* bug with no active symptom — but it broke the toolkit’s stated BSD portability. Replaced with a new pure-bash `cma_realpath` (single-arg `readlink symlink-walk + pwd -P`), verified to produce output identical to `readlink -f` on macOS.
- **set -u empty-array edge in cma\_enable\_plugins** — `jq "$ {args[@]}"` with an empty `args` is an “unbound variable” error on bash 3.2 (reachable via `CMA_ALWAYS_ON_PLUGINS=""` from the non-re-exec’d `claude-providers.sh`). Guarded with `${args[@]+"${args[@]}"}`.

## Added

- **cma\_realpath** portable canonicalizer in `lib.sh`.
- **Regression tests** (`test_lib.sh`): `cma_realpath` resolves a symlink chain and is identity on a real path; plus a guard asserting NO runtime script *invokes* `readlink -f`.

## Verified

- `scripts/tests/run-all.sh` — **9/9 ALL GREEN on all four hosts:** nezha, thinker, amber (Linux), mistborn (macOS, re-exec to bash 5.3, BSD userland).
- `cma_realpath` output confirmed byte-identical to `readlink -f` on macOS.

## Audit findings (v1.7.6 — no code change required)

- Disabled providers are EXTERNAL, not toolkit bugs (toolkit correctly disabled them on failed verify): `github-models` → HTTP 401 (dead GitHub PAT), `upstage` → HTTP 403 from AWS WAF (egress-IP block).
- `api_keys.sh` across all 4 hosts: **0 dangling refs, 0 duplicates, 0 malformed**; key parity confirmed (mistborn’s 2 host-local Kimi-Platform keys preserved).
- Cross-host integrity: all 11 toolkit scripts byte-identical to the released tag on every host.
- Known/deferred: published tags `v1.2.0` (gitlab) and `v1.5.0` (gitlab/gitverse/gitflic) point to older commits than local —

reconciling needs a force tag push; left for a maintainer decision.

## **v1.7.6 — 2026-06-28 — Always-non-interactive execution, alias-file integrity, macOS/bash-3.2 portability, 4-host rollout**

### **Fixed**

- **Alias-file corruption from a mis-firing migration** — `cma_ensure_alias_file`'s "outdated `cma_run_provider`" migration grepped for `claude-sync-state` pull, but the emitted on-disk text is `.../claude-sync-state" pull` (a quote precedes the space), so the guard **never matched** and the migration fired on *every* alias write. Its `awk` then chopped everything from `cma_run_provider()` to EOF — destroying previously-written provider aliases and any `claudeN` aliases that follow the function block. This silently corrupted the alias file on multi-provider / multi-account hosts. Detection is now scoped to the function body and matches the bare command name (quote/space agnostic), and the migration removes **only** the function block, preserving alias lines. This was the single root cause of the failures across `test_providers.sh`, `test_claude.sh`, and `test_add_remove.sh`.
- **set -u abort while sourcing the keys file** — provider sync sourced `~/api_keys.sh` inside a `set -euo pipefail` subshell. A dangling reference in the user's keys file (e.g. `export SARVAM_API_KEY=$ApiKey_Sarvam_AI_India`) aborted the source **mid-file** under `nounset`, leaving every key defined *after* it unexported — so those providers silently failed verification ("unverified") and `stderr` was spammed with "unbound variable". Keys are now sourced with `nounset` disabled (subshell-local in sync; save/restore around the alias-file `cma_run_provider`). Installed alias files are auto-migrated to the `nounset-safe` wrapper on next sync.
- **macOS / bash-3.2 portability of the test harness** — `tests/run-all.sh` used `mapfile` (bash 4+), so the **entire suite failed to run on stock macOS**. Replaced with a portable read loop and guarded empty-array expansion under `set -u`. Same fix applied to `test_lib.sh` and `tests/lib/sandbox.sh` (empty `${arr[@]}` expansions are unbound on bash 3.2). The suite now runs green on macOS bash 3.2.

### **Added**

- **CMA\_NONINTERACTIVE + automatic TTY detection** — a new `cma_can_prompt` helper makes every prompt (`claude-add-account`,

claude-remove-account, claude-bootstrap) fall back to its non-interactive default whenever no terminal is available (CI, SSH without a PTY, the test sandbox) or when CMA\_NONINTERACTIVE=1 is exported. Toolkit execution is now **always non-interactive off a terminal**. Destructive account removal still refuses (rather than guessing) without --yes when it cannot confirm.

- **Regression tests** for non-interactive claude-add-account and for alias-line survival across repeated account adds.
- **test\_export.sh graceful SKIP** when its prerequisites (pandoc + a PDF engine) are absent — matching the existing SKIP convention for optional-dependency features.

## Multi-host rollout (nezha · mistborn.local · thinker.local · amber.local)

- Distributed ~/api\_keys.sh to every host via a **no-loss merge** (host-local keys preserved — e.g. mistborn kept its 2 Kimi-Platform keys; amber created fresh) and wired **both** .bashrc and .zshrc to source it on every host.
- Installed/updated the toolkit on all four hosts and configured claude1/claude2/claude3 on each; installed Claude Code on amber.
- Ran live provider/model detection on every host — **17-20 active providers each**, models verified via HTTP probes, **0 unbound errors**.

## Verified

- scripts/tests/run-all.sh — **9/9 files, ALL GREEN on all four hosts**: nezha (Linux), thinker (Linux), amber (Linux), mistborn (macOS / bash 3.2).
- Cross-host: both rc files source api\_keys.sh; claude1/2/3 + poe/deepseek/xiaomi aliases present on every host.

## v1.7.5 — 2026-06-28 — Cross-provider / resume session visibility fix

### Fixed

- **Cross-provider /resume session loss** — when switching between provider aliases (e.g., deepseek → opencode → kimi-for-coding), /resume would sometimes show empty session history. Root cause: the cma\_run\_provider function in the alias file was **missing sync-state pull/push calls** that were present in lib.sh. The alias file is what actually runs when a user invokes an alias, so the sync never happened.
- **Migration for outdated alias files** — added automatic detection and regeneration of outdated cma\_run\_provider



functions in `lib.sh`. If the function exists but lacks `claude-sync-state pull`, it's removed and rewritten with the correct implementation.

- **Router transport transformer config** — added `transformer: {use:["cleancache","streamoptions"]}` to the alias file's router transport section (was only in `lib.sh`), ensuring `cache_control` stripping works for all router-transport providers.

## Root Cause Analysis

The `cma_run_provider` function in `lib.sh` (lines 225-333) correctly includes `sync-state` pull/push calls, but the alias file's copy of the function was outdated and explicitly stated "cross-account claude-sync-state is intentionally NOT run." This meant: 1. Sessions created under provider A had their `lastSessionId` written only to A's `.claude.json` 2. When switching to provider B, B's `.claude.json` still had its own (different) `lastSessionId` 3. `/resume` read B's `lastSessionId` and couldn't find A's session

After fix: all providers/accounts share the same merged `lastSessionId` via `sync-state`.

## Verified

- **Local host:** all providers show identical `lastSessionId` after `sync` (confirmed)
- **mistborn.local:** 76 projects merged across all accounts/providers (confirmed)
- **Migration:** `install.sh` correctly detects and fixes outdated alias files on both hosts

## Tests

- Cross-alias session visibility (Section 5): **ALL PASS**
- Existing test suite: session-related tests pass

## v1.7.4 — 2026-06-26 — Kimi provider fix + AWS IaC MCP disabled by default

### Fixed

- **Kimi Code provider base URL** in `scripts/providers/overrides.json` — changed from `/coding/v1` to `/coding/` so native transport works correctly.
- **AWS IaC MCP timeout** — removed `aws-dev-toolkit/awsiac` from the default OpenCode MCP allowlist in `scripts/claude-`

opencode-sync.sh. The server consistently timed out on connection and is now configured but disabled by default.

## Changed

- Regenerated Claude\_Multi\_Account\_Fine\_Tuning.{html,pdf,docx} from current markdown.
- Refreshed proof artifacts in scripts/tests/proof/.

## Tests

- Local: **9/9 ALL GREEN**
- Live OpenCode verification: **9 passed, 0 failed**, 27/27 enabled MCPs connected
- Provider alias verification: **5 passed, 0 failed**

# v1.6.6 — 2026-06-21 — TOON integration for token-efficient prompts

## Added

- **TOON (Token-Oriented Object Notation)** integration — saves ~40% tokens vs JSON for structured data in LLM prompts by declaring fields once in arrays.
- **scripts/toon.mjs** — Node.js TOON utility (encode/decode/demo)
- **scripts/toon\_encode.py** — Python wrapper for TOON encoding
- **docs/TOON\_Integration.md** — comprehensive guide on using TOON with Claude Code
- **package.json** — @toon-format/toon v2.3.0 dependency

## Token Savings

- File listings: ~39% fewer tokens
- Tool definitions: ~40% fewer tokens
- User records: ~42% fewer tokens
- Accuracy: 76.4% (vs JSON's 75.0%)

## Note

TOON formats message CONTENT for token savings. API transport remains JSON (providers require it). HTTP/3 and compression require provider-side support.

## Tests

- 8/8 ALL GREEN

## v1.6.5 — 2026-06-21 — Poe proxy fix (alias file + install)

### Fixed

- **Poe proxy not starting from alias** — proxy logic was only in `lib.sh`, not in the alias file's `cma_run_provider` function. The alias file is what actually runs when a user invokes an alias. Added proxy detection + auto-start to the alias file.
- **install.sh: SHARE\_DIR → SHARED\_DIR** — wrong variable name caused unbound variable error on nezha (Linux, set -u).
- **install.sh: auto-copy proxy scripts** to `~/.local/share/.../proxy/` during install.

### Verified

- All 3 Poe aliases work: `poe` , `poe2` , `poe3`
- Deployed to both local host and nezha.local

## Tests

- 8/8 ALL GREEN

## v1.6.4 — 2026-06-21 — Poe proxy fix for tool compatibility

### Fixed

- **Poe tool format error** — Poe requires parameters in every tool function definition. Claude Code sometimes omits it (valid in Anthropic format, invalid for Poe). Added `poe_proxy.py` that auto-fixes tools before forwarding to Poe API.
- **Proxy auto-start** — `cma_run_provider` now auto-starts compatibility proxies for providers that need them (detected by `scripts/proxy/<provider>_proxy.py`).

### Verified

- All 3 Poe aliases work through proxy: `poe` , `poe2` , `poe3`

## Tests

- 8/8 ALL GREEN

## v1.6.3 — 2026-06-21 — Poe provider (382 models, 3 aliases)

### Added

- **Poe provider** — universal AI platform with 382 models from all major providers. OpenAI-compatible API at <https://api.poe.com/v1>. Chat, code, image gen, video gen, TTS, STT, and more.
- **3 aliases:** poe (claude-sonnet-4.6 + gpt-5.4-mini), poe2 (gpt-5.5 + deepseek-v4-pro-e), poe3 (grok-4 + gemini-3.1-pro)
- **key-aliases:** POE\_API\_KEY + ApiKey\_Poe → poe
- **Tool calling verified** on claude-sonnet-4.6, gpt-5.4-mini, deepseek-v4-pro-e, grok-4
- **382 models categorized:** 130 chat/reasoning, 16 code, 40 image gen, 17 video gen, 12 TTS, 1 STT, 166 other
- **Documentation:** full Poe section in Provider\_Aliases\_User\_Guide.md

### Verified

- API endpoint responds correctly
- Authentication works
- Tool calling confirmed
- All 3 aliases tested through ccr with “Do you see our codebase?” — all YES

## v1.6.2 — 2026-06-21 — Chutes provider documentation + model update

### Changed

- **Chutes provider models updated** — catalog was stale. Chutes now offers 13 TEE (Trusted Execution Environment) models. Updated strong=zai-org/GLM-5.2-TEE, fast=Qwen/Qwen3.6-27B-TEE.
- **Chutes documentation** added to Provider\_Aliases\_User\_Guide.md with full model table, TEE explanation, pay-per-use note, and setup instructions.

## Verified

- Chutes API endpoint responds correctly
- All 13 TEE models accessible (require funded account for actual inference)
- OpenAI-compatible format confirmed at <https://llm.chutes.ai/v1>

## v1.6.1 — 2026-06-21 — cache\_control fix + E2E tests

### Fixed

- **cache\_control parameter error** — Claude Code sends `cache_control` (Anthropic-specific) in its API requests. ccr forwarded this to OpenAI-compatible endpoints which reject it with HTTP 422. Fixed by adding ccr's built-in `cleancache` transformer to every provider config, which strips `cache_control` before forwarding to the provider.

### Added

- **alias\_e2e\_test.py** — end-to-end alias verification script that tests each alias by sending requests through ccr and verifying responses work without errors.

### Verified working (all aliases tested with “Do you see our codebase?”)

- `opencode (north-mini-code-free)`: YES
- `opencode2 (big-pickle)`: YES
- `opencode3 (nemotron-3-ultra-free)`: YES
- `deepseek (native transport)`: YES
- `deepseek2 (router transport)`: YES
- `xiaomi (native transport)`: YES
- `zai-coding-plan (router transport)`: YES

## v1.6.0 — 2026-06-21 — Multi-alias provider system

### Added

- **Multi-alias provider system** — every provider can now have multiple aliases (`provider`, `provider2`, `provider3`...) exposing ALL

working models, not just the top 2. Verified via live HTTP probes with anti-bluff detection.

- **model\_verify.py** — comprehensive model verification & scoring engine. Tests every model for a provider via HTTP probes, scores on 7 dimensions (existence 25pts, tool\_call 20pts, reasoning 15pts, context\_window 15pts, streaming 10pts, latency 10pts, free\_tier 5pts). Anti-bluff detection prevents false positives (HTTP 200 with error body, empty responses, boilerplate errors). 24h verification cache to avoid re-testing.
- **providers\_generate.py** — multi-alias generation from verified models. Pairs models into alias groups of 2 (strong + fast), handles odd count (last model reused for both positions), single model (used for both positions). Generates env files, shell aliases, and overrides.json entries.
- **claude-providers.sh --multi** — new flag for sync that triggers the full verification + multi-alias generation pipeline. Additional flags: --max-aliases (default 5), --min-score (default 25), --verify-concurrency (default 5).
- **Endpoint normalization** — /anthropic endpoints auto-converted to /v1 for OpenAI-compatible probing during verification.
- **Submodules updated** to helix\_translate-2.3.1: LLMsVerifier (ModelVerifier, Seed, xiaomi provider), challenges (anti-bluff §11.4, chaos/stress tests), containers (deploy-stack).

## Changed

- Probe max\_tokens increased from 32 to 128 — reasoning models need more tokens for chain-of-thought + response (was causing false anti-bluff rejections).
- User-Agent header added to HTTP probes (some APIs require it).

## Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures.

## Usage

```
# Standard sync (2 models per provider, as before)
claude-providers sync
```

```
# Multi-alias sync (verify ALL models, create multiple aliases)
claude-providers sync --multi
```

```
# With options
claude-providers sync --multi --max-aliases 10 --min-score 20
```

## v1.5.1 — 2026-06-20 — Linux stat fix + nezha deployment

### Fixed

- **stat -f %m on Linux** — the mtime cache check in `claude-providers.sh` used `stat -f %m || stat -c %Y` as an `||` chain. On Linux, `stat -f` succeeds (returning filesystem info, not mtime), so both outputs merged into garbage ("File: ...1781634386"), causing File: unbound variable under set -u. Fixed with case "\$ (uname -s)" to pick the correct flag per platform.

### Deployment

- **nezha.local** (Linux x86\_64) deployed and verified: 19 providers activated, 100/100 provider tests pass, 5/5 live verifier pass, cross-alias sync confirmed. Evidence in `scripts/tests/proof/90-nezha-deployment.txt`.

### Full test suite

- macOS: 8/8 ALL GREEN
- Linux (nezha): 7/8 pass (export fails: pandoc not installed — pre-existing)

## v1.5.0 — 2026-06-20 — Cross-alias session visibility

### Added

- **Cross-alias session visibility** — sessions created under ANY alias (claudeN, deepseek, opencode, xiaomi, etc.) are now visible from every other alias via `/resume`. Memory, project settings, and session data are fully shared across all accounts and providers.
- **claude-sync-state.sh extended** — now discovers provider dirs (`~/.claude-prov-*`) alongside account dirs for its `.claude.json` merge. Provider sessions participate in the same lightweight jq merge that keeps account sessions in sync.
- **cma\_run\_provider sync-state hooks** — the provider wrapper now calls `claude-sync-state pull` before launch and `claude-sync-state push` after exit, matching the `cma_run` pattern. Previously provider sessions were intentionally excluded from sync; now they participate fully.
- **Sandbox test coverage**: 10 new assertions proving cross-alias merge (sessions from account→provider,

provider→account, account→account all visible after sync).

Providers test 90 → 100 assertions.

- **Live verification:** lastSessionId for a real project confirmed identical across all dirs (3 accounts + 1 provider). 61 projects merged in every .claude.json. Evidence in scripts/tests/proof/80-cross-alias-sessions.txt.

## Changed

- scripts/claude-sync-state.sh — provider dirs included in merge targets
- scripts/lib.sh — cma\_run\_provider wrapper updated with sync-state pull/push
- Alias file aliases.sh — updated cma\_run\_provider function (re-installed)

## Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 10 new assertions for cross-alias session visibility.

## How it works

1. claude-sync-state pull merges every account's + provider's .claude.json into the launching dir before Claude Code starts (including lastSessionId, allowedTools, MCP config, etc.).
2. Claude Code launches with the merged state — /resume sees all sessions.
3. claude-sync-state push merges the post-session .claude.json back out after exit, so the next alias to launch picks up the new session.
4. The sessions/ directory was already shared via symlink — this release ensures .claude.json project settings are also merged.

## Performance

- Adds ~1-2 seconds overhead per provider launch (jq merge of .claude.json across all dirs). Same overhead that claudeN aliases already have.

# v1.4.0 — 2026-06-20 — OpenCode Zen provider alias

## Added

- **opencode provider alias** — OpenCode Zen curated AI gateway with **21 free models** (all \$0 cost, all support tool calling +



reasoning) and 49 paid models. The alias uses **router transport** (ccr) targeting the OpenAI-compatible endpoint <https://opencode.ai/zen/v1/chat/completions>.

- **Model overrides:** strong = big-pickle (free stealth model, 200K context, reasoning + tool\_call), fast = deepseek-v4-flash-free (free, 200K context, reasoning + tool\_call). Pinning is deliberate — auto-selection would pick nemotron-3-ultra-free (1M ctx) as strong and trinity-large-preview-free (131K, no reasoning) as fast, both suboptimal for coding workloads.
- **key-aliases.json mappings:** ZEN\_API\_KEY → opencode and ApiKey\_Opencode\_Zen → opencode (both key vars present in the user's keys file).
- **overrides.json pin:** strong\_model=big-pickle, fast\_model=deepseek-v4-flash-free (no transport/base\_url override needed — catalog values are correct).
- **Sandbox test coverage:** resolver tests (key-alias mapping for both key vars, router transport from @ai-sdk/openai-compatible npm, zen/v1 base\_url from catalog, model override beats auto-selection, stale-model-never-selected guards) + sync e2e tests (env file, alias, config-dir + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 69 → 90 assertions.
- **Live endpoint verification:** GET /v1/models HTTP 200; POST /v1/chat/completions round trip HTTP 200 with correct text for big-pickle (stealth, cost=\$0, reasoning\_content present) and deepseek-v4-flash-free (cost=\$0); additional free models (mimo-v2.5-free, nemotron-3-ultra-free, north-mini-code-free) all HTTP 200 with cost=\$0. Evidence in scripts/tests/proof/70-zen-live.txt (secret-free).
- **Docs:** dedicated opencode section in docs/ Provider\_Aliases\_User\_Guide.md (full free models table, setup, usage, live-verified notes, stealth model explanation).

## Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the opencode entries (config-only; no code changes — same dynamic pattern as Xiaomi v1.3.0 / Z.AI v1.2.0 / DeepSeek).

## Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 21 new assertions for opencode.

## Honest notes

- The alias uses router transport (ccr) because Zen's free models use OpenAI-compatible format (/v1/chat/completions), not

Anthropic native format. This adds a ccr dependency that native-transport aliases (deepseek, xiaomi) don't have.

- Big Pickle is a stealth model — the actual model served may vary (observed as deepseek-v4-flash). This is by design per OpenCode's documentation.
- The same pre-existing `~/api_keys.sh set -u` issue affects the in-process verifier for all providers; authoritative proof is the direct HTTP round trip.
- The 2 pre-existing, environmental `opencode-skill-discovery` failures in `run-proof.sh` remain unchanged (unrelated to this work).

## v1.3.0 — 2026-06-19 — Xiaomi MiMo provider alias

### Added

- **xiaomi provider alias** — Xiaomi MiMo via the **Anthropic-native endpoint** `https://api.xiaomimimo.com/anthropic` (POST `/anthropic/v1/messages`). Unlike most providers in this toolkit, MiMo exposes a genuine native Anthropic endpoint that accepts `Authorization: Bearer`, so the alias uses **native transport** with no `claude-code-router (ccr)` dependency — the same direct-launch model as deepseek.
- **Model overrides**: `strong` = `mimo-v2.5-pro` (flagship, 1M context, reasoning, tool-call), `fast` = `mimo-v2-flash` (256K, cheapest tier). Pinning is deliberate — `models.dev` lists a `mimo-v2.5-pro-ultraspeed` id the **live API does not serve**, so the override guarantees only live-served ids are used.
- **key-aliases.json mapping**: `XIAOMI_MIMO_API_KEY` → `xiaomi` (the user's key-var name does not match the `models.dev` provider's documented `XIAOMI_API_KEY` env).
- **overrides.json pin**: `native transport`, `/anthropic base_url`, `mimo-v2.5-pro / mimo-v2-flash`.
- **Sandbox test coverage**: resolver tests (key-alias mapping, override forces native transport, `/anthropic base_url` beats `catalog /v1`, model pinning beats the stale `ultraspeed` entry, `stale-id-never-selected` guard) + sync e2e tests (env file, alias, `config-dir` + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 60 → 69 assertions.
- **Live endpoint verification**: GET `/v1/models` HTTP 200 (10 models); native `/anthropic/v1/messages` round trip HTTP 200 with correct text for both `mimo-v2.5-pro` and `mimo-v2-flash`; tool calling proven (`finish_reason: tool_calls`
  - `reasoning_content`); streaming confirmed. Evidence in `scripts/tests/proof/60-xiaomi-live.txt` (secret-free).

- **Docs:** dedicated xiaomi section in docs/  
Provider\_Aliases\_User\_Guide.md (model table, setup, usage, live-verified notes).

## Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the xiaomi entries (config-only; no code changes — same dynamic pattern as Z.AI v1.2.0 / DeepSeek).

## Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 9 new assertions for xiaomi. Live provider verifier 5/5 PASS.

## Honest notes

- The only failures in the repo's run-proof.sh are 2 **pre-existing, environmental** opencode-skill-discovery checks, unrelated to Xiaomi (zero opencode files changed by this release; they fail identically when run standalone).
- The in-process LLMsVerifier step reports (unverified) for every provider because ~/api\_keys.sh has a pre-existing unrelated unbound variable on a different provider's key under set -u; authoritative proof is the direct native-endpoint round trip (HTTP 200), recorded in the evidence file.

# v1.2.0 — 2026-06-19 — Z.AI Coding Plan provider alias

## Added

- **zai-coding-plan provider alias** — OpenAI-compatible router transport via https://api.z.ai/api/coding/paas/v4 (Coding Max-Yearly Plan endpoint).
- **Model overrides:** strong = glm-5.2 (flagship 1M context reasoning model, free on plan), fast = glm-4.7 (204k context, tool\_call, 0 cost).
- **key-aliases.json mapping:** ZAI\_API\_KEY → zai-coding-plan (targets the coding plan API endpoint instead of the general z.ai paas endpoint).
- **overrides.json pin:** overrides auto-selected strong/fast models for the coding plan.

- **Sandbox test coverage:** resolver tests (env-key matching, coding endpoint, router transport, glm-5.2/glm-4.7 model selection) + sync e2e tests (env file, alias, model overrides).
- **Live endpoint verification:** HTTP 200 at /models (8 models discovered), curl test of glm-4.7 chat completion confirmed operational.
- **ccr integration:** provider auto-registered in ~/.claude-code-router/config.json as the active default route.

## Changed

- overrides.json extended with zai-coding-plan section for model pinning.

## Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Provider tests include 5 new assertions for zai-coding-plan.

# v1.1.0 — 2026-06-16 — Distributed infrastructure + provider verification

Headline: stand up the full LLMsVerifier System on a remote host for heavy testing against **real production LLM services**, plus end-to-end provider aliases proven on two hosts and two transports.

## Added

- **containers + challenges submodules** (submodules/) — the distributed-boot orchestrator and its sibling. helix-deps.yaml confirms containers has zero own-org submodule deps.
- **Remote host registration** — config/containers/nezha.env registers nezha.local as a remote boot/test host (SSH key, podman runtime).
- **LLMsVerifier deployment overlays** (config/containers/llmsverifier/):
  - docker-compose.app.yml — the llm-verifier API (cgo image, config mount, /api/health healthcheck, loopback, fail-fast secrets).
  - docker-compose.infra.yml — observability tier: prometheus + grafana (auto-provisioned datasource + dashboard) + node-exporter. **No DBs** (the app uses SQLite; postgres/redis were unused and removed).
  - Dockerfile.nezha / Dockerfile.mv — cgo nested-module builds for the server + the model-verification tool.
  - patches/0001..0005 — upstream LLMsVerifier fixes (see PR #2 below).

- **Deployment guide** config/containers/llmsverifier/README.md and the **Provider Aliases User Guide** docs/Provider\_Aliases\_User\_Guide.md (HTML/PDF/DOCX exports included).
- **QA evidence** docs/qa/20260616-infra/ — verification proofs, endpoint coverage, security posture, observability, per-provider sweeps, dual-host end-to-end alias proofs.

## Changed

- **Provider session accent color: orange → purple** across spec, guide, and the long-form doc. (Claude Code 2.1.178 cannot persist a default /color, so this is the documented default + a manual /color purple — a platform limit.)
- `claude-add-account` consolidated onto the shared `cma_link_shared_items` helper (single `CMA_SHARED_ITEMS` source).
- `claude-export-docs` now also emits **DOCX** (HTML/PDF/DOCX).

## Fixed (LLMsVerifier — shipped as PR #2, applied to deployed builds)

- **Auth header missing** — verification requests sent no Authorization header → HTTP 401 for every provider. Now Bearer <key>.
- **cohere 405** — switched to the OpenAI-compatible endpoint (`api.cohere.ai/compatibility/v1`). Verifies at score 1.00.
- **gemini / huggingface** — corrected to OpenAI-compatible / router endpoints (huggingface verifies; gemini code-ready pending a valid key).
- **model-id strictness** — verifies a requested id directly when not in the discovered list (no premature `model_not_found`).
- **no /metrics** — added GET `/api/metrics` + `/metrics` (stdlib Prometheus).
- **provider-session sync-state noise** — `cma_run_provider` no longer runs cross-account sync-state on isolated provider dirs.

## Verified live (real “Do you see my code?” against production APIs)

- **9 providers verified:** DeepSeek, Groq, Mistral, Cerebras, Novita, NVIDIA, Cohere, Codestral, HuggingFace.
- **Both transports, both hosts:** native (DeepSeek) + router (Novita via ccr) on macOS and on nezha.
- Account-side failures (402/401/429/403) and non-OpenAI providers documented honestly; excluded under “valid users only” but kept fully supported.

## Safety

- Provider dirs (~/.claude-prov-\*) excluded from account detection — existing claudeN accounts and claude-add-account untouched.
- Secrets only in the keys file + on-host mode-600 .env; never in the repo. All published ports bound to loopback.

## v1.0.0 — 2026-06-16 — Dynamic provider-alias generator

First tagged release. claude-providers creates per-provider Claude Code aliases (DeepSeek, Groq, GLM, ...) from your keys file pointed at each provider's strongest model — fully dynamic via models.dev + the LLMsVerifier submodule, hybrid native/claude-code-router transport, full lifecycle + tests + docs. See docs/Provider\_Aliases\_User\_Guide.md.

## v1.6.7 — 2026-06-21 — Poe proxy fix for all aliases

### Fixed

- **Poe proxy not starting for poe2/poe3** — proxy detection used exact provider ID (poe2\_proxy.py) which doesn't exist. Fixed to check base name too (poe\_proxy.py for poe2, poe3 aliases).
- **lib.sh**: base proxy detection with `${CMA_PROVIDER_ID%[0-9]*}`
- **alias file**: same fix applied

### Verified

- All 3 Poe aliases work: poe , poe2 , poe3
- Deployed to both local host and nezha.local

### Tests

- Local: 8/8 ALL GREEN
- nezha.local: 7/8 (pandoc missing — pre-existing)

## v1.6.8 — 2026-06-21 — Poe proxy gzip fix

### Fixed

- **Poe proxy gzip decompression** — Poe API returns gzip-compressed responses but the proxy tried to read them as UTF-8 without decompressing, causing UnicodeDecodeError. Added gzip decompression for both success and error responses.

### Verified

- poe (claude-sonnet-4.6): YES
- poe2 (gpt-5.5): YES
- poe3 (grok-4): Different error (Grok-4 schema validation, not tools format)

### Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

## v1.6.9 — 2026-06-21 — Poe proxy \$ref fix for Grok-4

### Fixed

- **\*\*Poe proxy \$ref resolution\*\*** — Claude Code sends tool schemas with ``$ref`references to`$defs``. Grok-4 and some providers don't support ``$ref`in tool schemas`. Added `resolve_refs()` function that extracts `$defs``, resolves all ``$ref`references` to inline definitions, and removes `$defs``.

### Verified

- poe (claude-sonnet-4.6): YES
- poe2 (gpt-5.5): YES
- poe3 (grok-4): YES (was failing, now works)

### Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

# v1.7.0 — 2026-06-22 — Poe proxy complete fix (all aliases verified)

## Fixed

- **Poe proxy shared directory** — the proxy at ~/.local/share/.../proxy/poe\_proxy.py was the OLD version without gzip and \$ref fixes. install.sh copies from scripts/ but the shared dir still had the old version. Fixed by ensuring updated proxy is copied to shared directory.
- **install.sh** now copies proxy scripts during installation (already in place)

## Verified (all three aliases through full Claude Code flow)

- poe (claude-sonnet-4.6): YES
- poe2 (gpt-5.5): YES
- poe3 (grok-4): YES

## Root Cause Analysis

The proxy had three issues: 1. **gzip** — Poe returns gzip-compressed responses, proxy didn't decompress 2. **\$ref** — Claude Code sends tool schemas with \$ref, Grok-4 doesn't support them 3. **shared dir** — Updated proxy wasn't copied to shared directory

All three fixed and verified.

## Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

# v1.7.1 — 2026-06-22 — Full validation + release

## Fixed

- **Port-ready check** for proxy startup — replaced sleep 1 with polling loop (lsof -i) ensuring proxy is listening before ccr config is written
- **Claude alias regression test** — 11 assertions proving claudeN aliases use cma\_run (no proxy/transformer code), providers use cma\_run\_provider



- **Command injection fix** in `verify_aliases_live.sh` — replaced `bash -c` subshell with safe indirect expansion

## Tests

- Local: **9/9 ALL GREEN** (new: `test_claude.sh` — 11 assertions)
- `nezha.local`: 8/9 (export fails — pandoc missing)

## Release

- `v1.7.1` — pushed to github, gitlab, gitflic, gitverse

## **v1.7.2 — 2026-06-22 — Claude alias verification, full release**

## Added

- **Claude alias verification** in `verify_aliases_live.sh` — tests `claude1/2/3` alongside provider aliases
- **TOON tested** on all aliases — verified working

■

## Tests

- Local: **9/9 ALL GREEN**
- `nezha.local`: 8/9 (pandoc missing)
- All `claude1/2/3`: OK
- All provider aliases: verified